

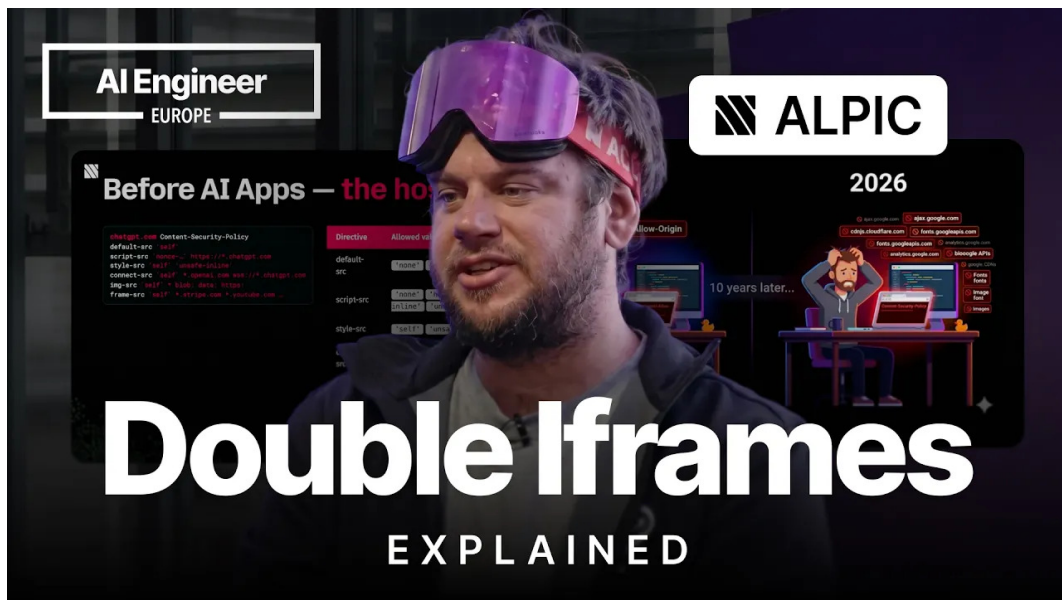
视频课程笔记

Why MCP and ChatGPT Apps Use Double Iframes

双层 iframe、CSP 与 MCP/ChatGPT Apps 的安全边界

Frédéric Barthelet, Alpic; 整理: Codex

2026-06-17



视频频道: AI Engineer

发布日期: 2026-06-15

视频时长: 00:20:11

视频链接: <https://www.youtube.com/watch?v=c-2eEv2ou7Y>

目录

1 为什么 MCP Apps 和 ChatGPT Apps 需要 UI 隔离	3
1.1 新的入口：从应用商店到对话中的发现	3
1.2 View 的生命周期：tool call、resource 与 iframe	5
1.3 DOM 里的意外发现：为什么会有 double iframe	5
1.4 CSP：浏览器给宿主页加上的第一层规则	6
1.5 本章小结	8
2 单层 iframe 的三条路线：srcdoc、same-origin 与 sandbox	8
2.1 从 CSP 的两条关键指令开始	8
2.2 iframe 是嵌套浏览上下文	9
2.3 srcdoc 的直觉方案与 script-src/nonce 冲突	10
2.4 放宽 CSP 后的同源风险：localStorage 与 cookies	11
2.5 sandbox 的收益与 allow-same-origin 的回弯	12
2.6 src 直连外部域名为何无法规模化	14
2.7 本章小结	15
3 Double iframe 架构：代理域、子域与 srcdoc 的组合	15
3.1 frame-src 的规模问题：无限应用域名清单	15
3.2 proxy-controlled domain：把外部域名压缩成受控入口	16
3.3 形式化模型：H -> I_o -> I_i	18
3.4 外层 iframe 与内层 iframe 的职责分工	19
3.5 按应用分配子域：避免 origin-indexed API 冲突	21
3.6 把应用 CSP 写入内层上下文	22
3.7 应用构建者的义务：metadata、依赖域名与可审计性	24
3.8 本章小结	25
4 开发者视角：CSP 元数据、Skybridge 与 CSP Inspector	25
4.1 开发者需要声明哪些域名依赖	25
4.2 为什么 CSP 调试容易像早期 CORS 一样难	26
4.3 Skybridge：类型安全、API polyfill 与开发工具	26
4.4 八球示例应用：从工具执行到视图渲染	27
4.5 CSP Inspector demo：从缺失域名到绿色通过	29
4.6 本章小结	30
5 总结与延伸：双层 iframe 的收益、成本与实践原则	30
5.1 产品原因：为什么对话产品要承载第三方 UI	31
5.2 浏览器机制：CSP、origin、sandbox 与双层 iframe	31
5.3 安全边界：可加载、可存储、可通信都要受控	32
5.4 运营规模：为什么代理域比无限域名清单更现实	32
5.5 开发者 workflow：从本地能跑到生产等价	32

5.6 开放风险：仍然容易踩坑的地方	33
5.7 本章小结	33

1 为什么 MCP Apps 和 ChatGPT Apps 需要 UI 隔离

1.1 新的入口：从应用商店到对话中的发现

这一讲的起点很具体：演讲者 Fred 是 Alpic 的 CTO 与联合创始人，Alpic 是一家 MCP hosting company。他要解释的是 ChatGPT 与 MCP 应用里出现的 double iframe 机制，以及这个机制对应用开发者意味着什么。这里的重点无需停留在开场寒暄，真正需要抓住的是产品形态的变化：MCP Apps（MCP 应用）与 ChatGPT Apps（ChatGPT 应用）把第三方产品和服务带进了通用对话代理。

在传统 Web 产品里，用户通常先进入网站或移动应用，再使用功能；在 ChatGPT、Claude 这类通用代理里，入口被重新分配了。应用既可以出现在 app store、connectors、应用目录这类可浏览入口里，也可以在对话上下文相关时被代理带入会话。例如，用户正在讨论旅行、数据分析、采购或客服流程时，相关应用可以进入对话，提供额外上下文、动作能力和产品流程。

这就是演讲者所说的两个核心条件之一：discoverability（可发现性）。对业务方而言，MCP Apps 与 ChatGPT Apps 是新的 surface area（产品表面）和 acquisition channel（获客渠道）；对宿主代理而言，它们是需要被发现、被调用、被展示的外部能力。

MCP Apps 与 ChatGPT Apps 的关键变化在于：第三方业务功能开始进入对话代理的主界面。应用既要被系统发现，也要在合适的时刻被用户看到、操作和信任。安全隔离的问题正是从这个产品需求自然长出来的。

第二个核心条件是 interactive UI（交互式界面）。早期的对话代理主要返回文本；应用进入之后，代理界面里会出现按钮、表单、图表、结果卡片、可交互面板等 UI 层。演讲者特别指出，这层 UI 可以由 MCP server 提供，也可能来自 generative UI。无论来源如何，宿主页面都需要回答同一个问题：来自第三方的可执行界面应当如何进入高权限的聊天页面？

这里还有一条标准化背景值得保留。讲者提到，这类 UI 入口先由前一场分享中的 MCP UI 思路推动，随后 OpenAI 在上一年 10 月通过 App SDK 发布，并逐步沉淀为 MCP 的第一个官方扩展：App Extension。这个脉络说明，double iframe 并非某个单点产品里的偶然实现细节；它服务的是一个正在跨宿主标准化的应用界面模型。换句话说，开发者面对的约束不只来自 ChatGPT，也来自 MCP Apps 在多个 client 中复用同一类 UI 契约的目标。

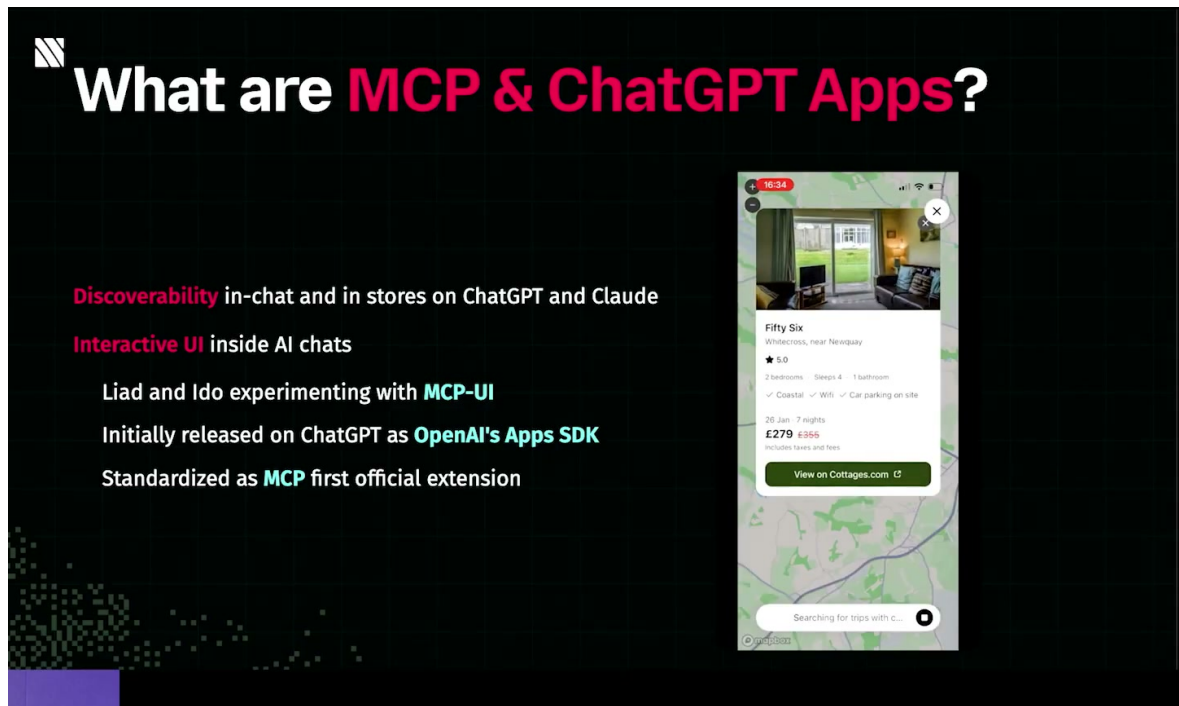


图 1: MCP Apps 与 ChatGPT Apps 的入口价值：应用可以在商店和对话中被发现，并把交互式 UI 带进聊天环境。¹

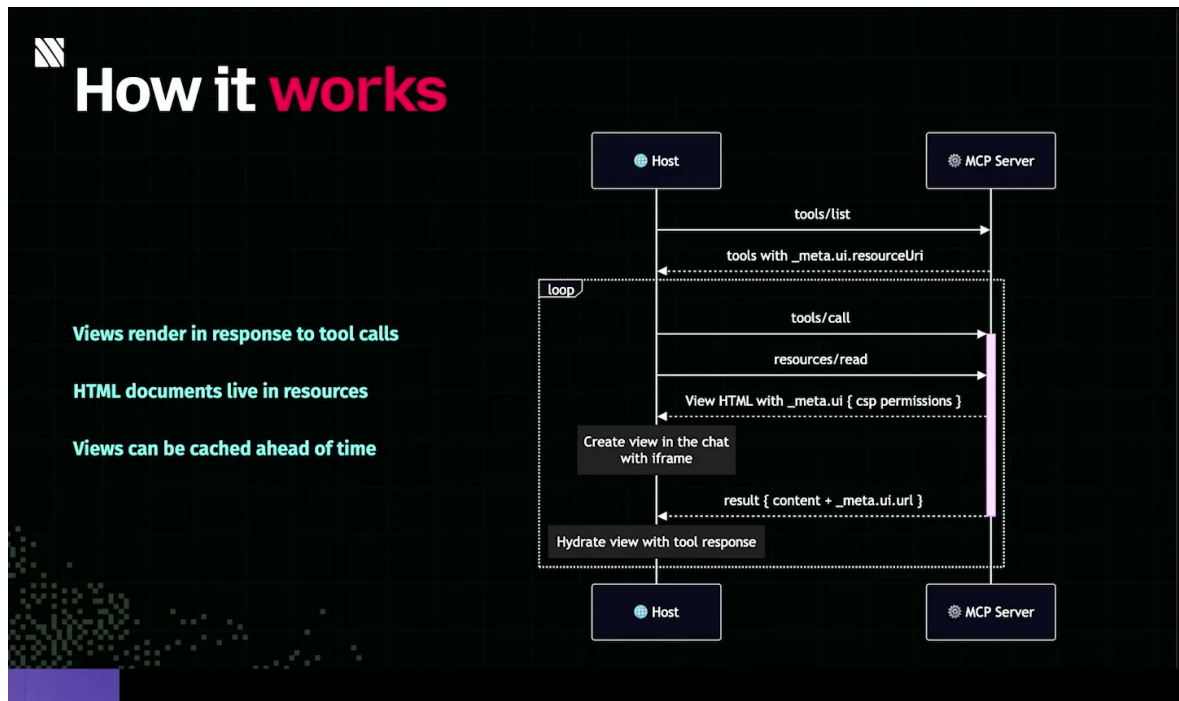


图 2: View 的生成链路：host 通过工具元数据发现 UI 资源，随后在工具调用后读取资源、创建 iframe，并把结果水合进可交互视图。²

¹视频画面时间：00:00:46–00:01:43。

²视频画面时间：00:02:02–00:03:31。

1.2 View 的生命周期：tool call、resource 与 iframe

演讲者随后把问题下沉到运行机制。对话中的小块 UI 被称为 view（视图）。在这里，view 指一段可以渲染在对话里的 HTML 界面；它通常包含 HTML、CSS 和 JavaScript。view 的出现并非孤立事件，它总是跟一次 tool call（工具调用）的结果绑定。

可以把整个过程看成一条链：

tool metadata → resource → tool call result → view in iframe

其中，tool metadata 表示工具在列表阶段声明的元数据；resource 表示 resource（资源），也就是渲染 UI 所需的 HTML/CSS/JS 文档或资源描述；tool call result 表示一次工具调用返回的数据；view in iframe 表示宿主把结果注入一个 iframe（内联框架）中，最终形成用户可见的交互界面。

这条链条里有一个容易被低估的细节：view 在工具调用完成前就已经进入宿主的视野。演讲者说，所有支持 UI 的工具会在对话开始阶段的 tool list call 中提前声明所需 resource。宿主如果支持 MCP Apps，就能提前知道某个工具适合用特定 UI 展示结果，也能提前缓存资源，或者在真正发生 tool call 时下载并提供这些资源。

可以把 tool list call 理解成一次“菜单同步”：MCP server 告诉宿主自己有哪些工具，其中哪些工具配有 view，渲染这些 view 需要哪些 resource。之后当用户触发某个工具时，宿主已经知道该把哪段 UI 放进对话。

因此，view 是 tool-call result UI：它不同于普通网页跳转，也不会把用户带离聊天窗口；它承担工具结果在会话中的交互式外壳角色。宿主创建 iframe，把 view 放进去，再把工具结果注入进去，动态内容由此显示给用户。

iframe 在这个阶段承担了第一层直觉上的隔离。它像宿主页面里嵌入的一个房间：用户看见它属于同一段聊天体验，浏览器却把它作为一个嵌套浏览上下文处理。这个“房间”可以拥有自己的 window、document 和资源加载过程。只要第三方 UI 开始运行 JavaScript，这个边界就从工程实现细节变成浏览器安全问题。

1.3 DOM 里的意外发现：为什么会有 double iframe

演讲者真正的疑问来自一次开发者工具观察。他打开宿主页面的 DOM，想看 ChatGPT 如何在对话里渲染第三方 UI，结果发现的结构比单层 iframe 更复杂：一个 iframe 里面还嵌着另一个 iframe，也就是 double iframe（双层 iframe）。

这种嵌套具有明确工程含义。对宿主而言，ChatGPT 页面本身持有用户会话、对话内容、宿主侧 localStorage、cookies、内部 API 调用能力等敏感资产。对第三方应用而言，view 又需要真实 UI 能力：脚本、样式、图片、网络请求、状态存储，甚至后续章节会讨论的按应用分配 origin。两边的目标天然有张力：宿主要允许应用“活起来”，同时又要阻断它触碰宿主页面的敏感区域。

为了给后续章节建立统一语言，可以先用最小模型描述这一层关系：

$$H \rightarrow I$$

其中, H 表示 host document (宿主文档), 例如 ChatGPT 页面; I 表示一个承载 view 的 iframe。单看这一行, 直觉上似乎足够: 宿主把应用 UI 放进 iframe。然而 DOM 中出现的 double iframe 说明真实设计更接近:

$$H \rightarrow I_o \rightarrow I_i$$

其中, I_o 表示 outer iframe (外层 iframe); I_i 表示 inner iframe (内层 iframe)。本章先把它作为一个观察事实和问题钩子: 为什么一个简单的 UI snippet 需要两层房间? 后续章节会展开单层 iframe 的三条路线为何都碰到边界, 以及双层结构如何把可运行性、隔离性和规模化运营拼在一起。

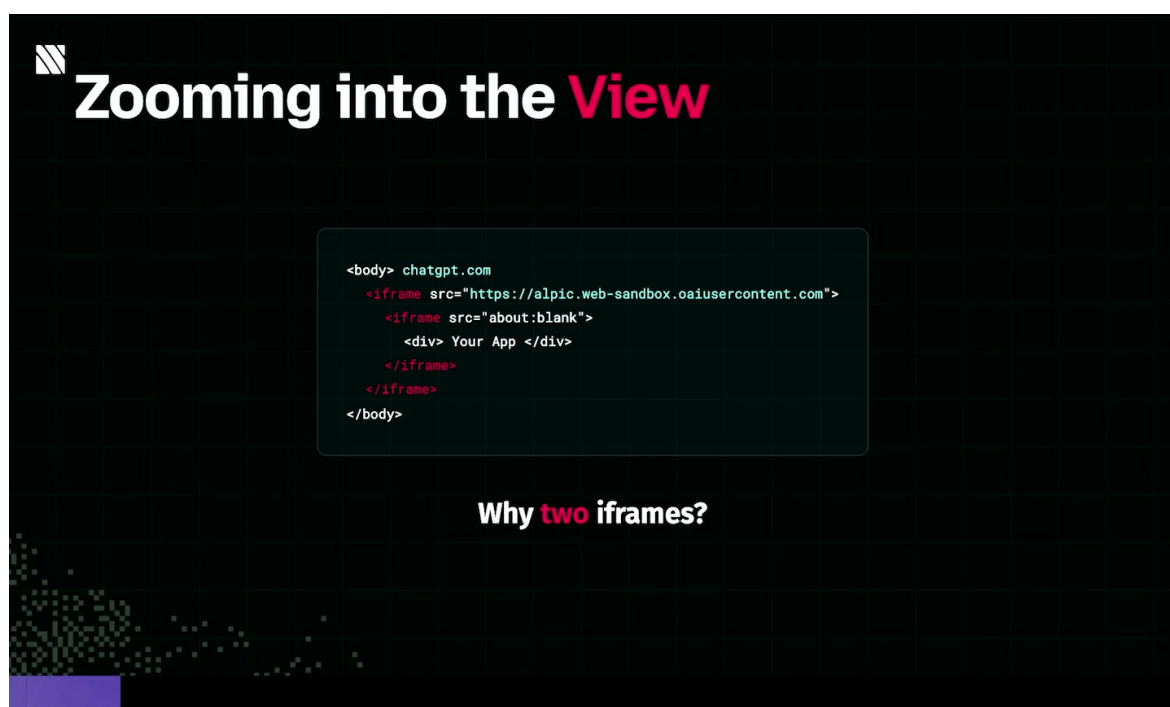


图 3: ChatGPT DOM 中的双层 iframe 线索: 外层 iframe 指向受控内容域, 内层 iframe 承载应用 UI, 这正是后文安全分析的对象。³

double iframe 的问题本质是: 第三方 view 既要像应用一样运行, 又不能像宿主页面的一部分那样获得宿主权限。两层 iframe 把“宿主接入”和“应用运行”拆成两个浏览器边界, 为后面的 CSP、origin、sandbox 和消息通道设计留下空间。

1.4 CSP: 浏览器给宿主页加上的第一层规则

在进入具体 iframe 方案之前, 演讲者先提醒听众看 ChatGPT 页面原本就带有的浏览器安全约束: Content Security Policy (CSP) (内容安全策略)。CSP 是服务器随文档响应返回的一组指令, 通常通过 HTTP response header 下发, 也可以在某些场景中以 meta policy 的形式出现。它告诉浏览器: 这个页面允许加载哪些脚本、样式、图片、iframe、网络连接, 哪些资源应被拒绝。

³视频画面时间: 00:03:31–00:04:16。

把 CSP 想成前台白名单会更直观。浏览器拿到宿主页面时，也拿到一张规则表。之后页面想加载脚本、图片、样式表、API 请求或 iframe 时，浏览器会对照规则表放行或拦截。一个紧凑的形式化表达如下：

$$\text{allow}(u, d, P) \iff u \in P[d]$$

其中， u 表示待加载资源的 URL 或 source expression； d 表示 CSP directive（指令），例如 `script-src` 或 `frame-src`； P 表示完整 CSP policy（策略表）； $P[d]$ 表示指令 d 对应的允许来源集合。公式的意思是：当资源 u 属于策略 P 在指令 d 下的允许集合时，浏览器才允许对应加载或执行。

演讲者在这一段没有展开所有 CSP 指令，只强调两个对后续结构最关键的指令：

- **frame-src**：控制哪些站点可以作为 iframe 被嵌入当前文档。
- **script-src**：控制哪些脚本可以在浏览器中执行。

这两个指令直接卡住了“把第三方 UI 放进 ChatGPT 页面”这件事。**frame-src** 关系到外部 view 能不能被嵌入；**script-src** 关系到 view 里的 JavaScript 能不能运行。到这里，产品需求已经变成浏览器安全设计题：第三方 UI 要进入对话，宿主页面的 CSP 又必须继续保护用户会话和宿主资产。

chatgpt.com Content-Security-Policy

```
default-src 'self'  
script-src 'nonce-...' https://*.chatgpt.com  
style-src 'self' 'unsafe-inline'  
connect-src 'self' *.openai.com wss://*.chatgpt.com  
img-src 'self' * blob: data: https:  
frame-src 'self' *.stripe.com *.youtube.com _
```

Directive	Allowed values
default-src	'none' 'self' https:
script-src	'none' 'nonce-...' 'strict-dynamic' 'unsafe-inline' 'unsafe-eval'
style-src	'self' 'unsafe-inline' 'nonce-...'
connect-src	'self' https: wss:
img-src	'self' data: blob:
frame-src	'self' 'none' blob:

图 4: 宿主页的 CSP 基线：frame-src 与 script-src 分别约束嵌入来源和脚本执行，是理解后续失败路线的前提。⁴

⁴视频画面时间：00:04:18–00:05:31。

CSP 的价值在于让浏览器成为执行规则的裁判。应用代码自身说“这段脚本安全”没有意义；宿主通过 CSP 给浏览器声明规则，浏览器按规则拦截资源加载和脚本执行。对于 ChatGPT 这样的宿主页面，CSP 是第三方 UI 进入之前已经存在的第一道门禁。

这里还要提前留意一个“未知未知”：MCP Apps 与 ChatGPT Apps 的 UI 隔离需要多层机制共同完成。CSP 管资源加载，iframe 管嵌套浏览上下文，origin 管 DOM 与存储边界，sandbox 管能力开关，消息通道管宿主与应用之间的数据交换。任何一层过宽，第三方 view 都可能从“可交互体验”滑向“宿主页面风险面”。

1.5 本章小结

本章只建立第一层模型。Fred 作为 Alpic 的 CTO 与联合创始人，把问题从 MCP Apps 和 ChatGPT Apps 的产品入口讲起：应用既要在商店或连接器生态中被发现，也要在聊天上下文里被唤起；它们还要提供交互式 UI，使对话代理从文本输出扩展到可操作的产品界面。

机制上，view 是与 tool call 结果绑定的 HTML/CSS/JS UI resource。工具通过 metadata 提前声明所需 view，宿主在工具调用后创建 iframe 并注入结果，让用户在对话里操作动态界面。

安全问题由此进入主线。开发者工具中观察到的 double iframe 说明，宿主并未把第三方 UI 当作普通页面片段处理。CSP 又进一步说明，ChatGPT 这类宿主页面从一开始就受 `frame-src` 与 `script-src` 等浏览器规则约束。下一章将接着回答：如果只用单层 iframe，为什么 `srcdoc`、`same-origin`、`sandbox` 与外部 `src` 会分别撞上不同的安全和可用性边界？

2 单层 iframe 的三条路线：srcdoc、same-origin 与 sandbox

2.1 从 CSP 的两条关键指令开始

讲者在这一段先把问题收束到浏览器安全模型上。内容安全策略（Content Security Policy，简称 CSP）可以理解成浏览器替宿主页面执行的一张“前台准入表”：哪些脚本能运行，哪些样式表能下载，哪些图片能加载，哪些 API 能连接，哪些页面能被嵌入，都要先通过这张表。

这里最关键的两条指令是 `frame-src` 与 `script-src`。`frame-src` 决定某个页面允许把哪些来源嵌进 frame；`script-src` 决定哪些脚本允许在浏览器里执行。用一个统一的形式可以写成：

$$\text{allow}(u, d, P) \iff u \in P[d]$$

其中， u 表示一个待加载的资源来源，例如某个脚本 URL 或某个 frame 的来源； d 表示 CSP 指令名，例如 `frame-src` 或 `script-src`； P 表示当前页面的 CSP 策略； $P[d]$ 表示策略 P 在指令 d 下列出的允许集合； $\text{allow}(u, d, P)$ 表示浏览器根据这条策略允许来源 u 通过指令 d 的检查。

CSP 的价值在于把“开发者想加载什么”变成“浏览器实际允许加载什么”。它像机场安检清单：乘客自己说要进登机口没有意义，清单里出现的证件、航班和入口才算数。对 ChatGPT 这类宿主页面来说，第三方 App 的 UI 代码进入页面前，先要经受这套清单约束。

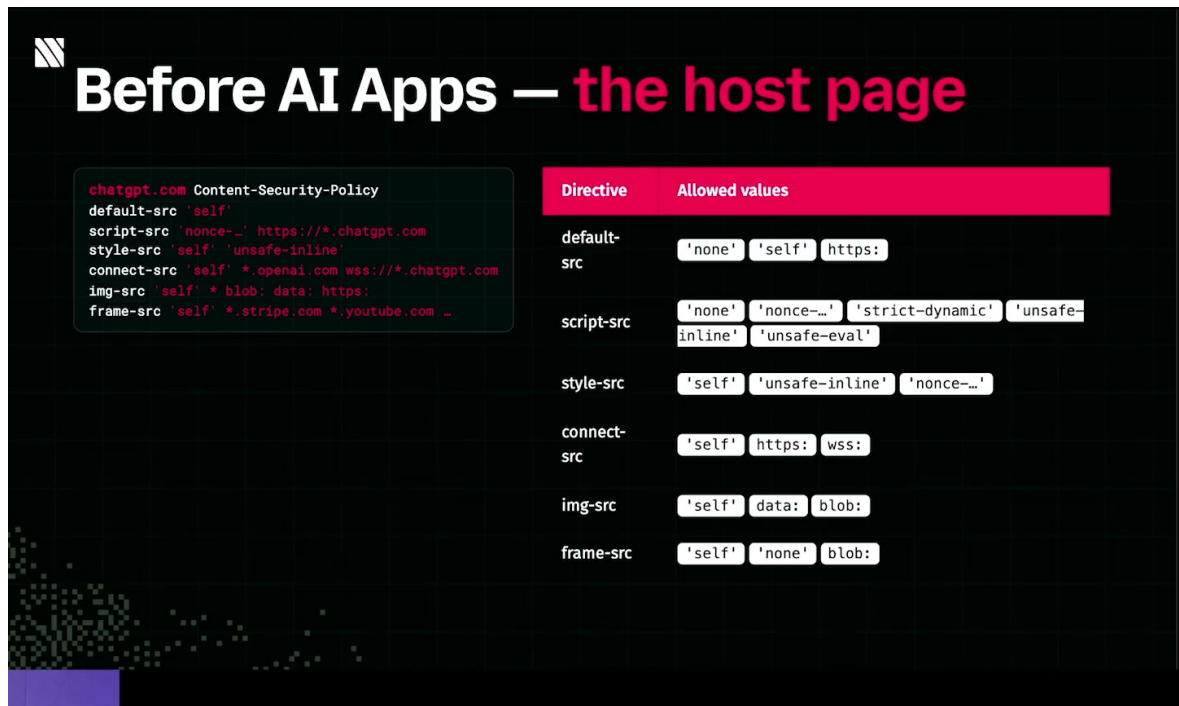


图 5: CSP 指令表把可执行脚本、可嵌入 frame、可连接 API 等能力拆成不同栏位，浏览器会按栏位逐项放行或拦截。⁵

2.2 iframe 是嵌套浏览上下文

为了把外部 UI 放进 ChatGPT，讲者接着引入内联框架（iframe）。iframe 是浏览器提供的 HTML 元素，用来在一个页面内部生成嵌套浏览上下文（nested browsing context）。直观地说，它像在主页面这栋建筑里开出一间小房间；更精确地说，它会产生自己的 window、document、加载过程和安全边界。

iframe 通常有两条加载路线。第一条是 **src 属性**：宿主页面给 iframe 一个 URL，浏览器去加载另一个页面，并把它渲染在 iframe 的矩形区域中。第二条是内联文档（**srcdoc**）：宿主页面直接把一段 HTML 内容塞进 iframe，让浏览器把这段内容当成 iframe 文档渲染。对 MCP 或 ChatGPT Apps 来说，**srcdoc** 看起来特别诱人，因为 MCP server 暴露出来的 resource 本来就是 HTML、CSS、JavaScript 组成的小型 View，把它直接注入 iframe 似乎路径最短。

⁵视频画面时间：00:04:18–00:05:31。



图 6: `srcdoc` 方案把 HTML 直接注入 `iframe`; 这种路线短, 然而它会继承父页面 CSP, 脚本执行会被 `nonce` 规则卡住。⁶

同源 (same-origin) 是理解后续失败路线的核心。浏览器给每个页面发一张安全“门牌”, 这张门牌由协议、主机和端口共同决定:

$$O = (\text{scheme}, \text{host}, \text{port})$$

其中, O 表示某个页面或 `frame` 的 origin; `scheme` 表示协议, 例如 `https`; `host` 表示主机名, 例如 `chatgpt.com`; `port` 表示端口, 例如 HTTPS 默认的 443。

两个文档是否同源, 可以写成:

$$\text{sameOrigin}(A, B) \iff O_A = O_B$$

其中, A 与 B 表示两个页面、文档或 `iframe`; O_A 表示 A 的 origin; O_B 表示 B 的 origin; $\text{sameOrigin}(A, B)$ 表示浏览器把 A 与 B 判定为同源。

本章的主线可以压缩成一句话: 单层 `iframe` 总是在“能运行 App 代码”和“别让 App 拿到宿主能力”之间摇摆。只用 `srcdoc` 会继承宿主的 origin 与 CSP; 放宽 CSP 会扩大宿主暴露面; 加 `sandbox` 会切断 origin 能力; 再加 `allow-same-origin` 又会把危险属性带回来。

2.3 `srcdoc` 的直觉方案与 `script-src/nonce` 冲突

第一条路线是直接使用 `srcdoc`。讲者的设想很自然: 既然 App resource 是一段纯 HTML, 那就把它放进 `iframe` 的 `srcdoc` 属性里, 让 ChatGPT 页面直接渲染这段内容。这样做的优势是无需额外 endpoint, 也无需让浏览器再去请求另一个页面。

⁶视频画面时间: 00:06:22–00:07:44。

问题出在 `srcdoc` 的继承关系上。当 `iframe` 通过 `srcdoc` 创建，并且没有额外的 `origin` 隔离时，它会共享父页面的 `origin`，也会受到父页面 CSP 的约束。也就是说，App 的脚本进入的是 ChatGPT 的安全制度之内，而 ChatGPT 的 `script-src` 非常严格：脚本必须带有宿主按请求预先生成的一次性脚本令牌（`nonce`）。

`nonce` 可以理解成宿主页面发出的“一次性工牌”。ChatGPT 页面自己的脚本拿得到这张工牌，第三方 App 注入的任意脚本拿不到。于是浏览器看到 App 的 JavaScript 时，会发现它缺少正确 `nonce`，进而阻止执行。结果是：HTML 可以显示一些静态内容，真正的交互逻辑无法跑起来。



图 7: `srcdoc` 的阻塞点：应用脚本缺少宿主页 `script-src` 所要求的一次性 `nonce`，浏览器会拦截脚本运行。⁷

这里的失败原因要落在 `script-src` 的 `nonce` 规则上：`iframe` 仍然可以显示内容，第三方脚本却会被浏览器拦住。对交互式 App 来说，脚本无法执行就等于 View 失去主要功能：按钮、状态更新、网络请求和 host API 适配层都很难正常工作。

2.4 放宽 CSP 后的同源风险：localStorage 与 cookies

讲者随后做了一个思想实验：如果为了让 App 代码能运行，放宽 ChatGPT 的 CSP，让它执行任意脚本，会发生什么？这个想法在工程上很危险，讲者也明确提醒这适合作为实验思路。

一旦 `srcdoc` `iframe` 继续共享宿主 `origin`，同时 `script-src` 又允许 App 脚本执行，App 脚本就获得了与父页面相同的 `origin` 身份。由于浏览器本地存储（`localStorage`）、`cookies`、`IndexedDB` 等能力都按 `origin` 索引，这个 `iframe` 内的脚本就可能读取宿主 `origin` 下的数据。讲者给出的具体风险是：第三方 App 可以读取 ChatGPT 的 `localStorage`，然后发送到自己的后端服务器。

⁷ 视频画面时间：00:06:22–00:07:44。

用同源公式表达，直接 `srcdoc` 的危险关系是：

$$\text{sameOrigin}(I, H) \iff O_I = O_H$$

其中， I 表示单层 `srcdoc` `iframe`； H 表示宿主页面，例如 ChatGPT； O_I 表示 `iframe` 的 `origin`； O_H 表示宿主页面的 `origin`。若 $O_I = O_H$ ，浏览器就会把它们看成同一安全身份下的文档。

放宽宿主 CSP 等于把“脚本执行问题”换成“宿主数据暴露问题”。这类风险的严重性很直接：`localStorage` 与 `cookies` 往往绑定登录状态、会话信息、用户设置或内部缓存。只要第三方脚本能以宿主 `origin` 运行，数据外传就从理论风险变成可写代码实现的路径。

2.5 sandbox 的收益与 allow-same-origin 的回弯

第二条路线是把 CSP 恢复到原状，然后给 `iframe` 加沙箱 (`sandbox`)。`sandbox` 是 `iframe` 的一个能力限制属性，可以拿掉脚本、表单、弹窗、同源身份等能力，再通过具体 `flag` 逐项恢复。讲者强调的关键点是：`sandbox` 可以让 `iframe` 进入不透明 `origin`，教学上可以近似记作空 `origin`：

$$O_S = \text{null}$$

其中， O_S 表示 `sandbox` 分配给 `iframe` 的不透明 `origin`；`null` 是教学上的简化记号，表示它不再等同于父页面 `origin`。

这样做确实解决了前面的同源访问风险。若 `sandbox` 让 `iframe` 变成 O_S ，并且 $O_S \neq O_H$ ，`iframe` 内脚本就无法像同源文档那样访问父页面 DOM、父页面 `localStorage` 或父页面 `cookies`。房间还在主建筑里，但门牌已经被换成一张无法匹配宿主门牌的临时牌。

代价也很硬：很多 Web API 依赖 `origin` 索引。`iframe` 变成 `null origin` 后，App 代码很难使用 `localStorage`、`IndexedDB` 和 `cookies`，因为浏览器找不到一个稳定的 `origin` 作为这些数据的归属。对真实 App 来说，这会伤害状态保存、缓存、认证跳转、用户偏好等基础体验。

于是自然出现第三步：给 `sandboxed iframe` 加 `allow-same-origin`。`allow-same-origin` 这个 `flag` 会把同源身份恢复给 `iframe`。问题在于，它恢复的恰好是前面要避免的危险性质：`iframe` 又回到父页面 `origin`，具备访问父页面 DOM、`localStorage` 与 `cookies` 的条件。讲者把这称为回到起点，因为它重新制造了“App 脚本在宿主 `origin` 下运行”的局面。



图 8: 放宽 CSP 后暴露的同源风险: srcdoc 视图继承父页面 origin, 脚本可能触达宿主 cookies、localStorage 与 DOM。⁸

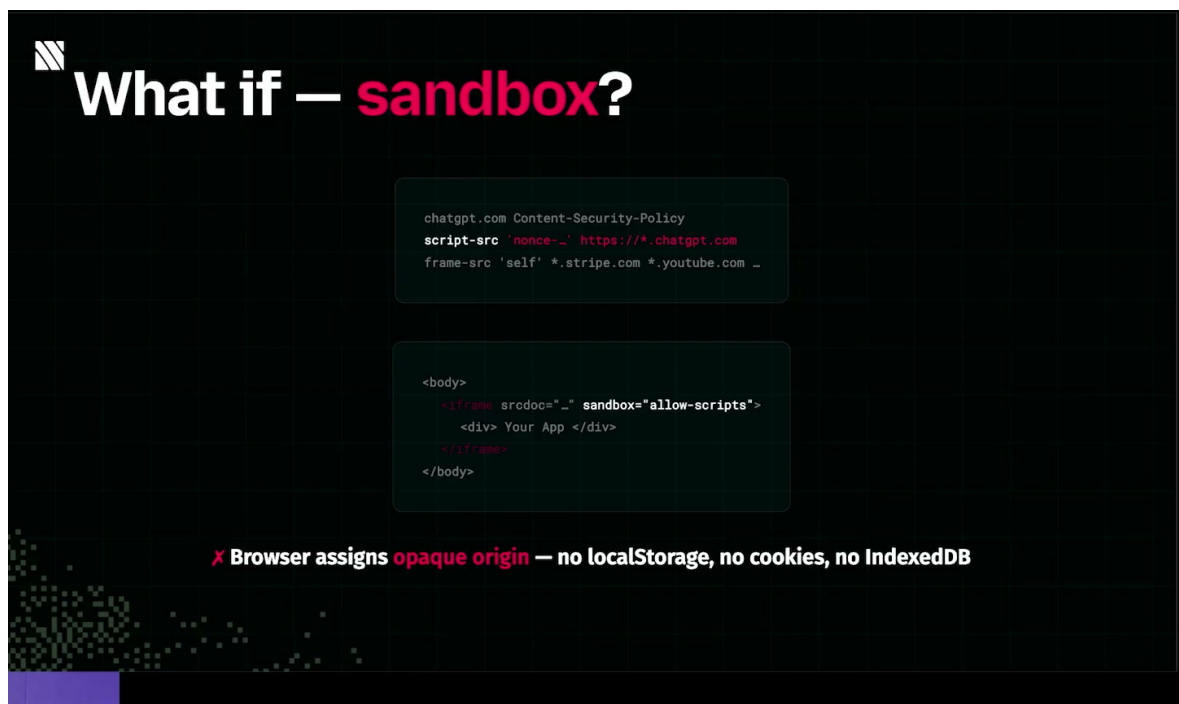


图 9: sandbox 会把 iframe 分配到 opaque origin; 隔离增强的同时, localStorage、cookies 与 IndexedDB 等按 origin 索引的能力也会失效。⁹

⁸视频画面时间: 00:07:44–00:08:22。

⁹视频画面时间: 00:08:22–00:09:12。

sandbox 的核心矛盾是能力开关的粒度。去掉同源身份，安全性提升，App 的 origin-indexed 能力会塌掉；加回 **allow-same-origin**，App 能力恢复，宿主隔离又被削弱。这个回弯正是单层 **iframe** 很难同时满足安全与可用性的原因。

2.6 src 直连外部域名为何无法规模化

第三条路线转向 **src** 属性。与 **srcdoc** 直接注入 HTML 相比，**src** 会让 **iframe** 加载一个正常 URL。对 App 开发者来说，这看起来也很顺：把自己的 View 做成服务器上的一个 endpoint，例如 **/view**，然后让 ChatGPT **iframe** 指向这个 endpoint。

这条路表面上绕过了 **srcdoc** 的继承问题，因为 App 页面来自开发者自己的服务器，可以拥有自己的 origin。真正的瓶颈来到 **frame-src**：宿主页面必须在 CSP 里列出允许被嵌入的 frame 来源。若每个 MCP App 或 ChatGPT App 都用自己的域名提供 View，OpenAI 就需要把所有 App 开发者的域名加入 ChatGPT 的 **frame-src** 白名单。

从策略形式看，这意味着：

$$\text{allow}(u, \text{frame-src}, P) \iff u \in P[\text{frame-src}]$$

其中， u 表示某个 App View endpoint 的来源；**frame-src** 表示 CSP 中控制 **iframe** 嵌入来源的指令； P 表示 ChatGPT 宿主页面的 CSP； $P[\text{frame-src}]$ 表示允许被嵌入为 **iframe** 的来源集合。

当 App 数量从几个变成几千、几万时，这个集合会失控。每新增一个 App，就可能新增一个域名；每个域名还涉及审核、更新、撤销、通配符边界、子域名接管、证书与运营故障。讲者在这一段把问题推进到下一章的核心：需要一种方式，把无限增长的外部 endpoint 压缩到宿主可控、可审计、可规模化的一小组入口。



图 10: **src** 方案的 **frame-src** 困境：如果每个 MCP App 都用自己的 view endpoint，宿主必须维护难以扩展的 **iframe** 域名白名单。¹⁰

这里的规模化问题不只是“白名单太长”。CSP 白名单是浏览器安全边界的一部分，写得过宽会增加攻击面，写得过窄会让合法 App 失效。平台型系统真正需要的是稳定入口、可撤销路由、按 App 分离的 origin，以及能把开发者声明转成浏览器可执行策略的基础设施。

2.7 本章小结

本章沿着讲者的顺序检查了单层 iframe 的三条路线。第一，直接 `srcdoc` 最省事，却继承宿主 origin 与 CSP，因 ChatGPT 的 `script-src` nonce 规则而无法执行第三方 App 脚本。第二，放宽宿主 CSP 能让脚本跑起来，同时把 App 脚本放进宿主同源环境，localStorage、cookies 与父页面 DOM 都会进入风险范围。第三，`sandbox` 能制造 null origin 来隔离宿主，可是会让 localStorage、IndexedDB、cookies 等 origin-indexed 能力失效；加 `allow-same-origin` 又会恢复前面的危险条件。最后，`src` 指向外部 endpoint 虽然让 App 获得自己的 origin，却把压力转移到 `frame-src` 域名白名单，平台规模一大就难以管理。

这些失败路线共同说明：真正的问题超出了“怎样把 HTML 塞进 ChatGPT”这个层面，关键在于怎样在脚本执行、origin 身份、CSP 白名单和平台规模之间同时取得可运行、可隔离、可运营的平衡。下一章的 double iframe 架构，正是为了解开这个多重约束。

3 Double iframe 架构：代理域、子域与 srcdoc 的组合

3.1 frame-src 的规模问题：无限应用域名清单

上一章已经把单层 iframe 的几条路线推到了边界：直接用 `srcdoc` 会继承父页面的安全上下文；放宽脚本策略会碰到 nonce 与同源能力；把每个 View 作为外部 `src` endpoint 又会触发宿主主页的嵌入白名单问题。本章要回答的核心问题是：当 MCP 应用越来越多时，宿主如何在保持隔离的同时，还能把未知第三方 UI 渲染到对话产品里。

这里先看 `frame-src` (`frame-src`)。它是 Content Security Policy (内容安全策略，Content Security Policy，简称 CSP) 里控制“哪些来源可以被嵌入为 iframe”的指令。若每个 MCP 应用都把自己的 HTML View 暴露在自家域名，例如 `https://app-a.example/view`、`https://app-b.example/view`、`https://vendor-c.example/widget`，宿主页面 *H* 的策略就必须不断追加这些域名：

$$\text{allow}(u, d, P) \iff u \in P[d]$$

其中：

- *u* 表示准备加载的资源 URL 或来源；
- *d* 表示 CSP 指令名，例如 `frame-src`、`script-src`、`connect-src`；
- *P* 表示宿主或视图使用的 CSP 策略表；
- $\text{allow}(u, d, P)$ 表示浏览器根据策略 *P* 判断 *u* 是否允许出现在指令 *d* 对应的加载位置。

¹⁰视频画面时间：00:09:37–00:10:30。

问题来自规模:应用商店每增加一个 MCP 应用,宿主就要把新应用的域名加入 $P[\text{frame-src}]$ 。如果应用数量为 N ,白名单至少跟着 N 线性增长;如果每个应用还有开发、预发、生产、客户专属域名,实际条目数量会变成

$$|P[\text{frame-src}]| \approx \sum_{i=1}^N k_i$$

其中 k_i 是第 i 个应用需要暴露的可嵌入域名数量。这个模型在早期样例中看起来清楚,一进入开放生态就会失控。更糟糕的是,CSP 是浏览器强制执行的边界;配置一旦漏掉,iframe 直接无法渲染。配置过宽,又会把宿主页面暴露给来源不明的嵌入面。

为什么直接暴露每个应用域名难以规模化 若每个 View 都作为自己的 app-domain endpoint 被 ChatGPT 这类宿主直接嵌入,宿主的 `frame-src` 就要追踪所有第三方应用域名。开放生态下这等价于维护一张持续增长、难以审计、容易遗漏的外部域名清单。讲者把这个压力点作为进入 double iframe 架构的真正动机:问题首先是规模化运维问题,其次才是浏览器 API 技巧问题。

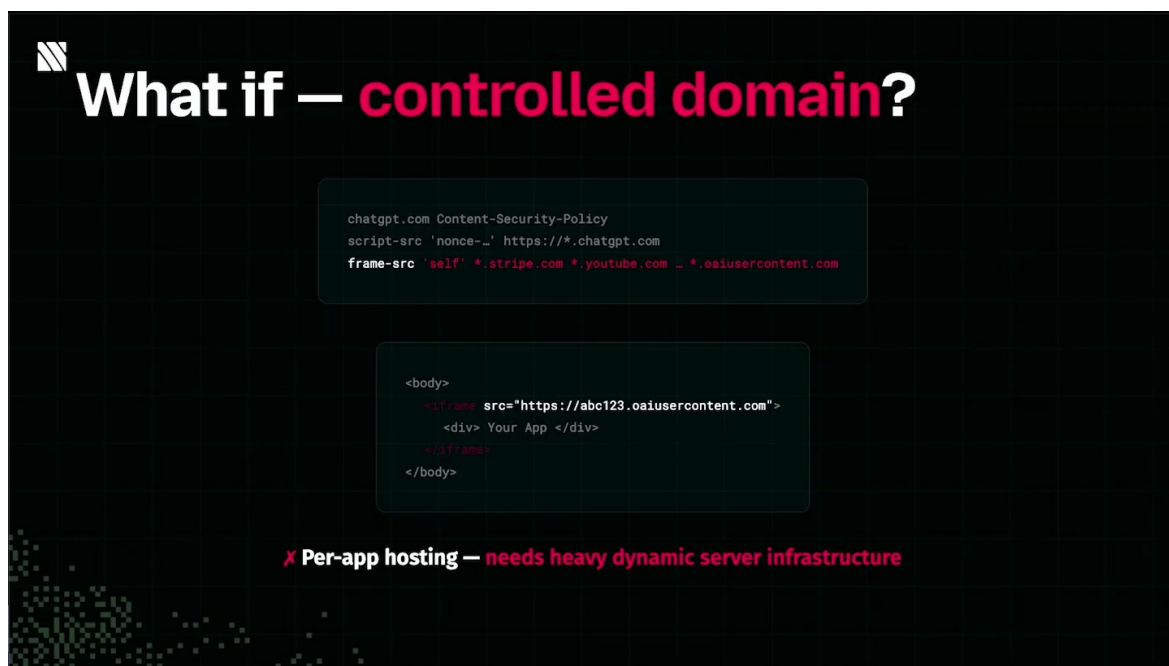


图 11: 受控域名方案把 `frame-src` 收敛到平台域名,由代理层按子域或路由键转发到具体应用资源。¹¹

3.2 *proxy-controlled domain*: 把外部域名压缩成受控入口

讲者给出的第一步是引入代理受控域名(proxy-controlled domain)。以 `openaiusercontent.com` 为例,它是宿主控制的用户内容域。宿主只需要在自己的 `frame-src` 中允许这个稳定域名,浏览器看到的 iframe 来源就会落在宿主可管理的范围内:

¹¹视频画面时间: 00:10:48–00:11:29。

$$P_H[\text{frame-src}] = \{\text{https://}^*.openaiusercontent.\text{com}\}$$

其中 P_H 是宿主页面 H 使用的 CSP。直觉上，这个域名像一个“统一收发室”：外部应用来自不同公司，包裹入口却统一经过宿主认可的门牌。浏览器只需要确认门牌在白名单里，后续路由由宿主基础设施处理。

把这个想法落到实现里，宿主需要一个受控服务器：它从 MCP server 获取应用资源 R ，也就是 View 所需的 HTML、CSS、JS，再把资源映射到 `openaiusercontent.com` 之下的路径或子域。例如，子域第一段可以作为 routing key：

$$\text{https://abc123.openaiusercontent.com/loader} \longrightarrow R_{\text{abc123}}$$

这里的 `abc123` 是应用或资源的路由键。它把“无限外部域名”折叠成“一个受控域名加多个内部路由键”。在 `frame-src` 视角，宿主的允许列表稳定；在应用视角，每个应用仍然能找到自己的资源。

CSP 白名单像浏览器前台登记簿 CSP 可以理解为浏览器前台的一本登记簿。访客要进入某个区域，需要名字出现在对应栏位里：`frame-src` 管 `iframe` 访客，`script-src` 管脚本访客，`connect-src` 管网络请求访客。`proxy-controlled domain` 的作用，是让前台只登记一个受控入口，再由入口内部完成应用级路由。这样减少了宿主 CSP 的全局 churn，也让审计对象更集中。

不过，单靠代理域还没有解决全部问题。若宿主真的在自己的受控域名上直接提供所有第三方 HTML，就会进入一个很尴尬的位置：宿主域名开始承载自己并不了解的第三方 UI 代码。讲者指出，这意味着宿主要为未知代码的行为、依赖、网络访问和渲染副作用承担更多基础设施责任。对 OpenAI 或 Anthropic 这样的平台尚且是沉重负担；对资源较少的宿主而言，动态托管所有应用 View 的成本会很现实。

代理域仍需避免直接承载任意第三方代码 受控域名降低了 `frame-src` 的复杂度，却会把未知第三方 UI 推到宿主基础设施之上。若宿主直接把任意应用 HTML 当作一等页面服务出去，攻击面会集中到代理域：路由错误、缓存污染、宽泛 CSP、跨应用存储共享、日志泄露都可能变成平台级问题。后面的 double `iframe` 机制就是为了缩小这个承载面。

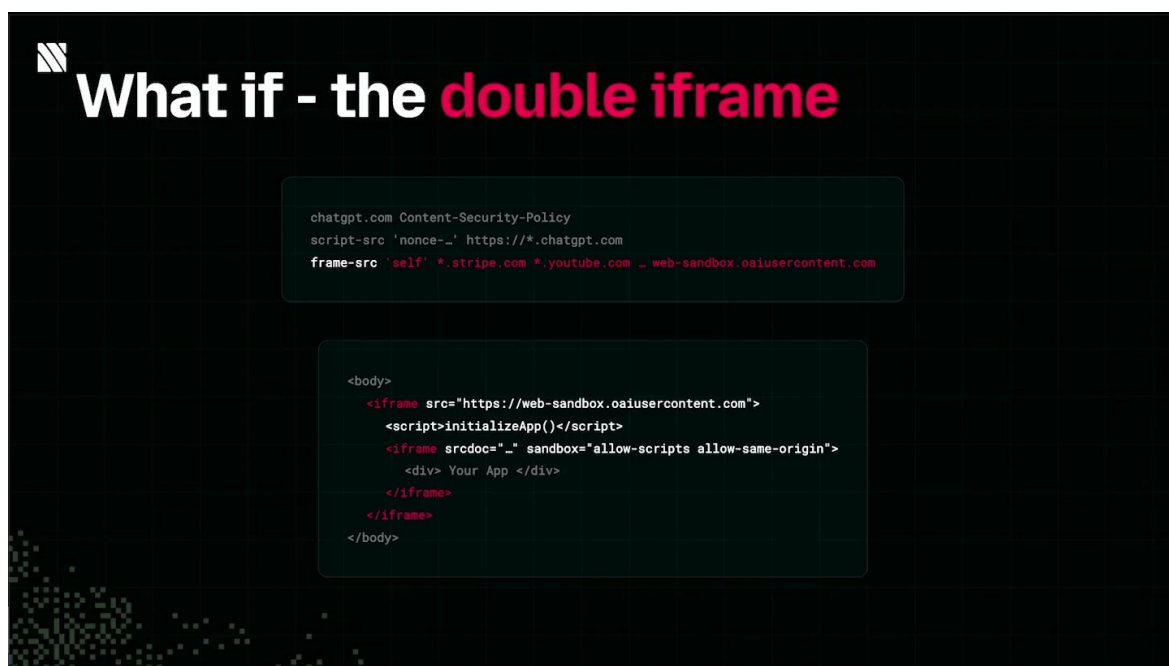


图 12: double iframe 的核心结构: 宿主页加载外层受控 iframe, 外层脚本再用 `srcdoc` 与 `sandbox` 承载应用 HTML。¹²

3.3 形式化模型: $H \rightarrow I_o \rightarrow I_i$

double iframe (双层 iframe) 把上面的压力拆成两层浏览器上下文:

$$H \rightarrow I_o \rightarrow I_i$$

这组符号在本章中统一含义如下:

- H : Host document, 宿主文档, 例如 ChatGPT 主页面, 拥有宿主 DOM、会话、cookies、localStorage 和对话上下文;
- R : App resource, MCP server 暴露出来的应用 View 资源, 通常包含 HTML、CSS 和 JS;
- I_o : outer iframe (外层 iframe), 由 H 加载, 来源是代理受控域名;
- I_i : inner iframe (内层 iframe), 由 I_o 创建, 使用 `srcdoc` (srcdoc) 承载应用 HTML;
- O_H : 宿主 origin, 例如 $O_H = \text{https://chatgpt.com}$;
- O_P : 代理受控 origin, 例如 $O_P = \text{https://abc123.openaiusercontent.com}$;
- O_S : sandbox 产生的不透明 origin, 可用 $O_S = \text{null}$ 作教学近似;
- P : CSP 策略表, 描述 `frame-src`、`script-src`、`connect-src` 等指令允许哪些来源;
- M : 消息通道, 通常对应 `postMessage` (postMessage) 一类受控跨窗口通信边界。

¹²视频画面时间: 00:12:07-00:12:33。

浏览器里的 origin 可以写成：

$$O = (\text{scheme}, \text{host}, \text{port})$$

两个页面同源的判断是：

$$\text{sameOrigin}(A, B) \iff O_A = O_B$$

double iframe 的关键关系是：

$$O(I_o) = O_P, \quad O_P \neq O_H$$

也就是说，宿主 H 嵌入的是代理域下的 I_o ，这个外层页面与宿主页面分属不同 origin。 I_o 再加载一段共享 loader script（加载器脚本，loader script），由加载器恢复或获取 R ，随后创建 I_i ，把应用 HTML 放进 **srcdoc**。由于 I_i 的父上下文是 I_o ，应用代码的 origin 关系围绕 O_P 展开，脱离 O_H 的宿主身份。

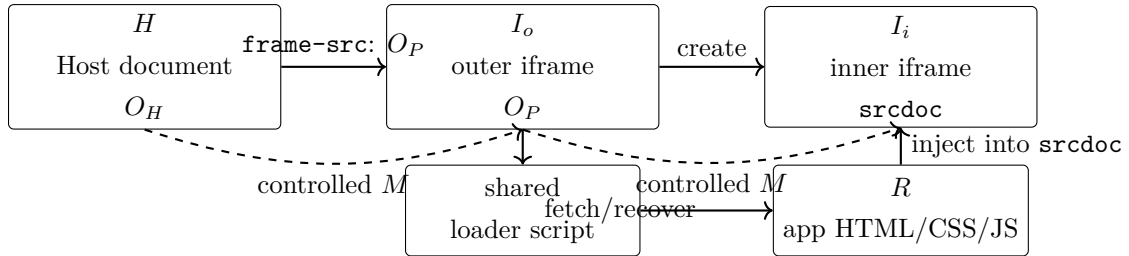


图 13: double iframe 的基本形状：宿主 H 只直接嵌入外层代理 iframe I_o ，外层加载器再创建内层 **srcdoc** iframe I_i 来运行应用资源 R 。虚线表示需要显式校验的消息通道 M 。

为什么采用这组结构，直接让每个应用提供页面又会输在哪里？答案可以压缩成三点。第一， H 的 **frame-src** 面向 O_P ，避免无限追加外部应用域名。第二， I_o 的内容可以是平台统一服务的加载器，宿主无需把任意第三方 HTML 当作外层页面直接托管。第三， I_i 承载真正的应用 View，安全策略可以在更细粒度的上下文里重写和应用。

double iframe 的安全不变量 本章的核心不变量是 $O_P \neq O_H$ 。应用 View 可以获得足够的浏览器运行环境，却不应获得宿主页面的 origin 身份。 $H \rightarrow I_o \rightarrow I_i$ 的分层让宿主 CSP、代理域路由、应用 **srcdoc**、每应用子域和消息通道 M 各自承担一部分边界责任。

3.4 外层 iframe 与内层 iframe 的职责分工

外层 iframe（outer iframe） I_o 的职责偏平台化：它由宿主允许的代理域加载，运行同一份 loader script，并负责恢复应用资源、创建内层 iframe、桥接宿主与应用之间的有限消息。讲者强调，这个外层内容可以对所有应用保持一致。平台服务的是“加载器页面”；每个应用的任意 HTML 进入内层 View。

内层 iframe（inner iframe） I_i 的职责偏应用化：它通过 **srcdoc** 接收应用 HTML，也就是 R 的主体。由于 **srcdoc** 在浏览器中默认继承父上下文 origin，前一章提到的风险仍然存在；这一次

父上下文从 H 变成了 I_o 。这正是外层 `iframe` 的价值所在：即使 `srcdoc` 继承 `origin`，它继承的是代理域 O_P ，隔离于宿主 `origin` O_H 。

为了让内层应用代码能执行， I_i 通常还要配合 `sandbox` (`sandbox`) 属性和 `allow-same-origin` (`allow-same-origin`) 标志。直觉上，`sandbox` 是能力遮罩，默认拿走脚本、表单、弹窗、同源身份等能力；标志再按需恢复其中几项。对 `srcdoc` 来说，关键分支可以写成：

$$O(I_i) = \begin{cases} O_P, & \text{with } \texttt{sandbox} \text{ and } \texttt{allow-same-origin} \\ O_S, & \text{with } \texttt{sandbox} \text{ and without } \texttt{allow-same-origin} \end{cases}$$

其中：

- $O(I_i) = O_P$ 表示内层应用在代理域身份下运行，可以使用依赖 `origin` 的能力，例如 `localStorage`；
- $O(I_i) = O_S$ 表示内层应用得到不透明 `origin`，很多按 `origin` 索引的 API 会失去正常语义；
- 危险点来自父 `origin` 的选择：若父上下文是 O_H ，`allow-same-origin` 会重新贴近宿主敏感身份；若父上下文是每应用代理子域 O_P ，风险被限制在应用自己的代理空间里。

为什么 `allow-same-origin` 在这里仍然有用 很多真实 UI 需要 `localStorage`、`cookie`、`IndexedDB` 或某些依赖 `origin` 的浏览器行为。完全不透明的 O_S 像临时房间，没有稳定门牌，应用很难保存状态。`double iframe` 的做法是先把房间搬离宿主大楼，再给应用自己的门牌 O_P 。这样应用能工作，宿主身份仍然被隔开。

外层和内层之间还会存在消息通道 M 。视频这一段主要讲 `iframe` 结构与 CSP，消息边界更多是由架构推导出来的工程事实：宿主、加载器、应用 View 必须交换初始化参数、尺寸、工具结果、用户动作或 `host API` 调用。浏览器中常见手段是 `postMessage`。这条通道需要按安全边界处理；普通事件总线的心智模型过于粗糙。

`postMessage` 的剩余风险 M 应当同时校验 `origin` 和 `payload`。接收方至少要检查 `event.origin` 是否来自预期的 O_H 或 O_P ，检查 `event.source` 是否匹配对应窗口，并用 `schema` 校验消息类型、字段和值域。若只校验消息名，恶意应用、被污染依赖或错误嵌入页面都可能伪造 `host API` 调用。这个风险在字幕里只是被 `iframe` 通信需求间接暗示，实际实现里必须显式处理。

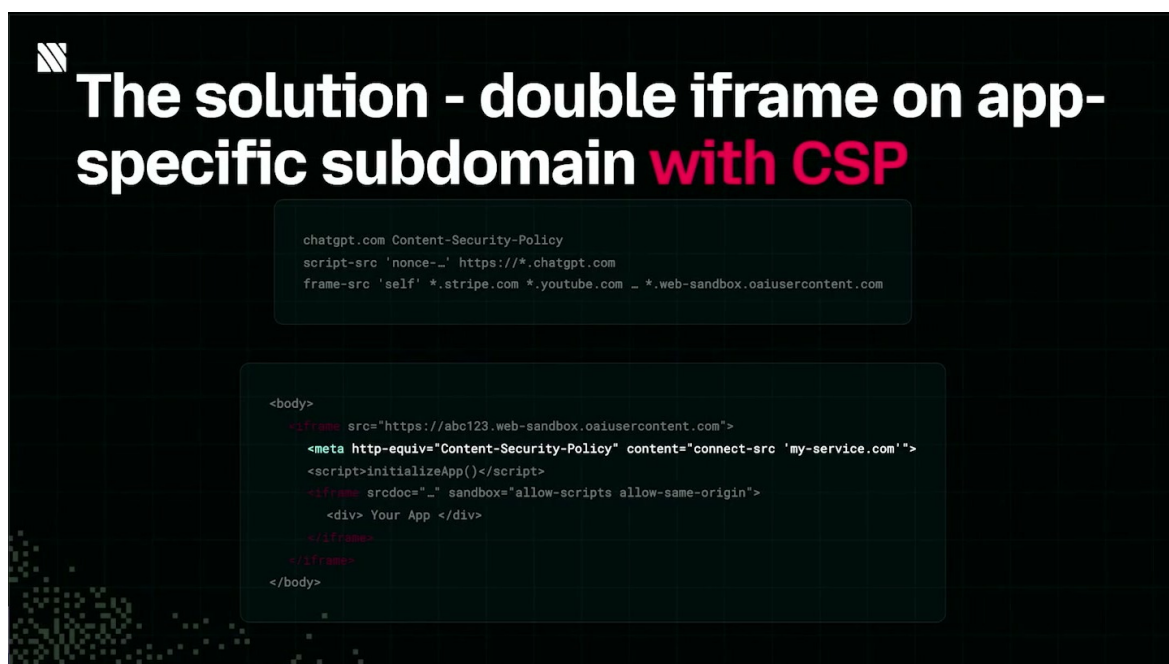


图 14: 每应用子域与 CSP meta 组合在一起: 应用获得稳定 origin, 存储边界按子域分离, 资源加载能力由应用级 CSP 约束。¹³

3.5 按应用分配子域: 避免 origin-indexed API 冲突

讲者在 00:12:36–00:13:17 进一步指出, 外层加载器页面虽然内容相同, 也应该部署到不同子域。原因是浏览器里大量能力按 origin 索引: localStorage、IndexedDB、cookies、Cache Storage、部分权限状态, 都与 $O = (\text{scheme}, \text{host}, \text{port})$ 绑定。

若所有应用都共享同一个代理 origin, 例如:

$$O_P = \text{https}://\text{apps.openaiusercontent.com}$$

那么应用 A 和应用 B 在浏览器看来住在同一个门牌下。应用 A 写入的 localStorage key, 应用 B 可能看到同一命名空间; cookie 路径和域设置一旦处理粗糙, 也可能互相污染。讲者用 ABC123 与 ABC456 举例: ABC123 应用不应读取 ABC456 应用的本地状态。

因此更合理的形状是每个应用使用一个子域:

$$O_{P, \text{ABC123}} = \text{https}://\text{abc123.openaiusercontent.com}$$

$$O_{P, \text{ABC456}} = \text{https}://\text{abc456.openaiusercontent.com}$$

两者满足:

$$O_{P, \text{ABC123}} \neq O_{P, \text{ABC456}}$$

¹³视频画面时间: 00:12:37–00:13:39。

这使得同源规则天然隔开两套 origin-indexed API。工程上，宿主可以对每个子域服务同一份 loader script，只把子域第一段作为 routing key。这样，基础设施复杂度比“为每个应用托管一份完整页面”低很多，浏览器隔离效果却更接近每应用独立运行环境。

每应用子域的收益 每应用子域把“同一份加载器脚本”与“不同应用 origin”结合起来。平台复用代码，浏览器负责隔离。只要路由键、证书、CSP 和缓存策略处理得当，`abc123.openaiusercontent.com` 与 `abc456.openaiusercontent.com` 会自然拥有不同的本地存储命名空间。

3.6 把应用 CSP 写入内层上下文

double iframe 还需要处理第三方 View 自己能加载什么。宿主页面 H 的 CSP 管的是宿主；代理外层 I_o 的 CSP 管的是加载器；真正应用 UI 在 I_i 中运行，它也需要自己的 Content Security Policy (Content Security Policy)。讲者提到 MCP spec 提供了一种声明方式，应用开发者可以在 metadata 中列出依赖域名，平台再把这些声明重写成 View 可用的 CSP。视频画面里这一步通过外层 app-specific iframe 文档中的 `meta http-equiv="Content-Security-Policy"` 完成，位置在加载器脚本和内层 `srcdoc` iframe 旁边；这样浏览器会在应用视图运行前拿到应用级资源边界。

这一步的本质是把开发者意图变成浏览器可执行策略。一个应用若要调用外部 API，就应该在 metadata 里声明 `connect-src`；若要加载第三方脚本，就声明 `script-src`；若要展示远程图片，就声明 `img-src`；若要再嵌入别的 frame，就声明 `frame-src`。这些声明最后进入 P_i ，也就是内层应用上下文的策略表。

可以把这张应用级策略表写成一个更直观的映射：

- $P_i[\text{connect-src}]$ 映射到允许访问的 API domains；
- $P_i[\text{script-src}]$ 映射到允许加载的 script domains；
- $P_i[\text{img-src}]$ 映射到允许展示的 image domains；
- $P_i[\text{frame-src}]$ 映射到允许继续嵌入的 frame domains。

其中 P_i 只描述应用 View 的允许列表。它无法替代宿主 P_H ，也无法替代消息通道 M 的验证。把层次分清非常重要： P_H 让 H 只嵌入可信代理域； P_o 约束外层加载器； P_i 约束应用资源； M 约束跨窗口数据流。

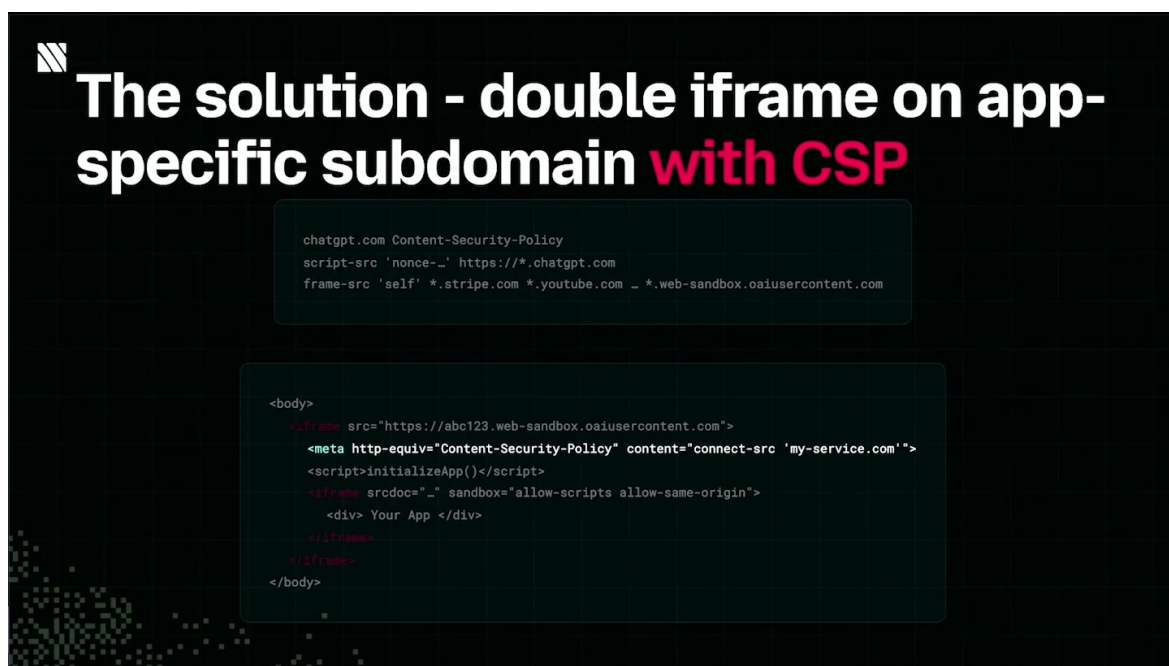


图 15: 最终方案把应用 CSP 写入内层上下文, 让应用声明的脚本、网络、图片和嵌套 frame 来源进入浏览器强制执行路径。¹⁴

开发者最容易漏掉的 CSP 细节 应用能在开发模式中加载成功, 并不意味着生产环境一定成功。开发者若漏报 `connect-src`, API 请求会被浏览器挡掉; 漏报 `script-src`, 脚本加载或执行会失败; 漏报 `img-src`, 图片可能静默消失; 漏报 `frame-src`, 二级嵌入会直接被拒绝。CSP 问题的坏处是表现常常像“网络坏了”或“组件没渲染”, 根因却在策略表里。

讲者还提醒, 这套方案并非全新发明。早期 Facebook app marketplace 也面对过相同问题: 平台应用需要把第三方 UI 放进自己的产品上下文里, 同时平台又不愿把宿主身份、用户会话和跨应用状态完全暴露给第三方代码。MCP Apps 与 ChatGPT Apps 只是把这个老问题带进了 LLM agent 与工具调用的新场景。

¹⁴视频画面时间: 00:12:37-00:13:39。

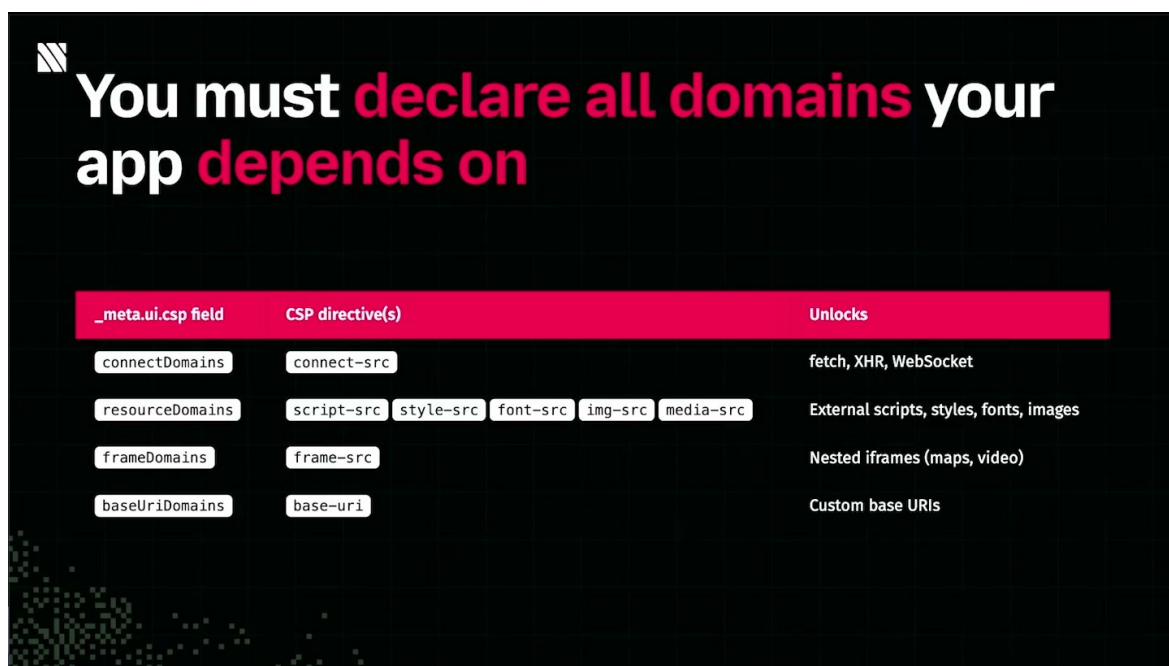


图 16: MCP 应用元数据中的 `_meta.ui.csp` 字段会映射到浏览器 CSP 指令，例如 `connect-src`、`script-src` 和 `frame-src`。¹⁵

3.7 应用构建者的义务：metadata、依赖域名与可审计性

从应用开发者视角，本章最硬的要求是：构建 MCP / ChatGPT Apps 时，必须把 View 依赖的外部域名写入应用 metadata。metadata 属于运行期安全输入，它会影响最终被重写进内层 iframe 的 CSP 指令。少写一个域名，浏览器就可能在运行期拒绝加载；多写一个宽泛通配符，应用就扩大了自己的攻击面。

可以把构建者义务拆成五类：

- 网络请求：所有后端 API、遥测 endpoint、认证回调域名应进入 `connect-src`；
- 脚本执行：所有远程脚本来源应进入 `script-src`，并尽量减少动态脚本依赖；
- 图片和媒体：图片 CDN、头像域、图表导出服务应进入 `img-src` 或对应媒体指令；
- 再嵌入内容：若 View 还要嵌入其他 iframe，需要声明 `frame-src`；
- 基础文档策略：根据平台能力声明 `style-src`、`font-src`、`base-uri` 等约束，避免默认过宽。

CSP 与早期 CORS 调试的相似痛点 讲者把 CSP 的开发体验类比成早期跨源资源请求调试：开发者看到请求失败，最初很难判断是代码 bug、网络错误、服务端响应、浏览器策略，还是 metadata 漏写。CORS 关注“这个网页能不能读另一个 origin 的响应”；CSP 更像“这个文档能不能加载、执行、连接或嵌入某个来源”。两者都把浏览器变成了安全执行者，错误信息通常需要专门工具才能快速定位。

¹⁵视频画面时间：00:14:00–00:14:48。

还需要补充一个经常被忽视的风险：metadata 的粒度会塑造未来审计能力。若开发者为了省事写成 `https://*.example.com` 或允许过多第三方 CDN，安全团队在审查应用时就很难判断真实依赖链。更稳妥的做法是按功能列出最小域名集合，并把每个域名映射到业务用途：哪个域提供 API，哪个域提供脚本，哪个域提供图片，哪个域提供嵌入内容。这样的 metadata 才能支撑后续的自动检查、商店审核和生产问题排查。

3.8 本章小结

本章从 `frame-src` 的无限白名单问题出发，解释了为什么开放式 MCP / ChatGPT Apps 生态不能简单依赖“每个应用暴露自己的 View endpoint”。proxy-controlled domain，例如 `openaiusercontent.com`，把宿主 CSP 的嵌入入口收束到一个受控域；但直接托管未知第三方 UI 会让宿主承担过重的基础设施和安全责任。

double iframe 的机制把责任拆开：宿主 H 嵌入代理域下的外层 iframe I_o ， I_o 运行共享 loader script，恢复应用资源 R ，再创建内层 `srcdoc` iframe I_i 。通过每应用子域， O_P 与 O_H 分离，应用之间的 origin-indexed API 也被隔开；通过 metadata 生成的 CSP，应用 View 自己的 `connect-src`、`script-src`、`img-src`、`frame-src` 能进入浏览器强制执行的策略表。最后，任何跨层通信 M ，尤其是基于 `postMessage` 的通道，都要保留 origin、source 与 payload 校验。架构给了边界，工程实现仍然要逐项守住这些边界。

4 开发者视角：CSP 元数据、Skybridge 与 CSP Inspector

前一章已经把双层 iframe 的结构讲清楚：宿主页 H 先加载外层 iframe I_o ，外层 iframe 位于代理受控域 O_P ，再由 I_o 创建内层 iframe I_i 来承载应用资源 R 。从浏览器安全模型看，这套结构很合理；从开发者每天调试应用的角度看，真正的痛点才刚开始。讲者在 00:15:00 之后把问题说得很直接：跨源资源安全与 CSP 配置很容易配错，体验像早期 CORS 一样折磨人。

4.1 开发者需要声明哪些域名依赖

在 MCP / ChatGPT Apps 的工作流里，应用代码通常由 MCP server（MCP 服务器）暴露工具、资源与渲染说明；widgets（小组件）和 views（视图）负责把工具结果变成可交互 UI。为了让宿主能够安全地渲染这些 UI，应用还要提供 metadata（元数据），其中包含视图将访问的外部域名与资源类型。

这里的核心关系可以压缩成一个很朴素的检查：

$$\text{allow}(u, d, P) \iff u \in P[d]$$

其中：

- u ：视图尝试访问的 URL 或 origin；
- d ：CSP 指令，例如 `connect-src`、`img-src`、`style-src`、`script-src`、`frame-src`；
- P ：由宿主、代理层和应用 metadata 共同生成的 CSP 策略；
- $\text{allow}(u, d, P)$ ：浏览器最终是否允许这次加载或连接。

这条公式的工程含义很硬：如果视图实际访问了某个 API 域名，而 metadata 没有把它列进 `connect-src`，开发者本地可能看不出问题，生产环境里的浏览器会直接拦截请求。类似地，远程图片对应 `img-src`，CSS 来源对应 `style-src`，脚本来源对应 `script-src`，嵌套 frame 来源对应 `frame-src`。

metadata 是上线契约 应用 metadata 里的 CSP 域名清单是一份上线约束，不能当作文档备注。它会进入宿主的实际安全策略，决定视图在生产环境中能否发请求、加载图片、加载样式、执行脚本或嵌入 frame。开发者遗漏一个域名，结果通常是生产环境功能失效或应用商店提交被拒。

一个常见误判是把 CSP 当成“部署最后补一下的 header”。在 MCP / ChatGPT Apps 场景中，CSP 更像应用能力声明：工具返回什么、视图会渲染什么、视图需要访问哪些域，都要提前让宿主知道。宿主负责保护用户会话、对话上下文和自身 DOM，因此它有充分理由要求这份清单精确。

4.2 为什么 CSP 调试容易像早期 CORS 一样难

讲者把 CSP 痛点类比为 CORS，是因为两者都具备同一类工程摩擦：策略由浏览器执行，错误经常在“真正的宿主环境”里才暴露。开发者在普通本地页面、独立 storybook 或宽松 dev server 中调通了请求，并不能证明它能在 ChatGPT 这类宿主里工作。

演讲者提到，当时 OpenAI 的 developer mode（开发者模式）曾经为了让开发更顺滑而移除所有 CSP。短期看，这会让本地开发更少报错；长期看，它会制造一种危险的错觉：代码在开发环境运行正常，进入生产或提交应用商店时，缺失域名才被浏览器拦住。

developer mode 遮蔽 CSP 会制造生产事故 如果开发环境移除了 CSP，测试覆盖就缺少最关键的一层约束。请求能够发出，只能说明应用逻辑大致可运行；它无法证明 metadata 覆盖了真实资源依赖。生产失败的根因常常很普通：某个新 API、CDN、图片域名或分析端点没有进入 CSP 清单。

这类问题还有一个隐蔽点：视图访问的域名不一定只来自开发者手写的 `fetch`。依赖库可能加载字体，图片组件可能走图片代理，错误监控 SDK 可能上报遥测，OAuth 流程可能跳到另一个授权域。只靠代码审查找全这些域名，成本高且容易漏。

4.3 Skybridge：类型安全、API polyfill 与开发工具

Alpic 为这个开发痛点做了一个开源框架 Skybridge（Skybridge）。讲者将它描述为建立在官方 App SDK（应用开发工具包）和 App Extension（应用扩展）之上的功能超集，目标是把应用开发中容易散落的类型、宿主 API 差异和调试工具收束到一条更稳的开发链路里。

Skybridge 的第一类能力是端到端类型安全：MCP server 输出的工具定义、工具结果、widgets 和 views 之间的数据形状可以在同一套类型系统里对齐。直觉上，它像把后端 API schema、前端 props 和工具调用结果放进同一份合同：服务器说会返回什么，视图就按同一份类型消费什么，很多低级错配会在开发期暴露。

第二类能力是 host API polyfills（宿主 API polyfill）。不同宿主对 App SDK / App Extension 的支持边界可能不同，某些 API 可能是 ChatGPT 特有能力，某些能力又存在于别的 host。polyfill 的作用是把些差异包装成更稳定的开发接口，减少同一个应用在不同宿主中分叉实现的压力。

第三类能力是开发工具。讲者重点展示的是 CSP Inspector（CSP 检查器），它专门处理本章的核心问题：metadata 声明了哪些域，视图运行时实际访问了哪些域，两者之间是否有缺口。

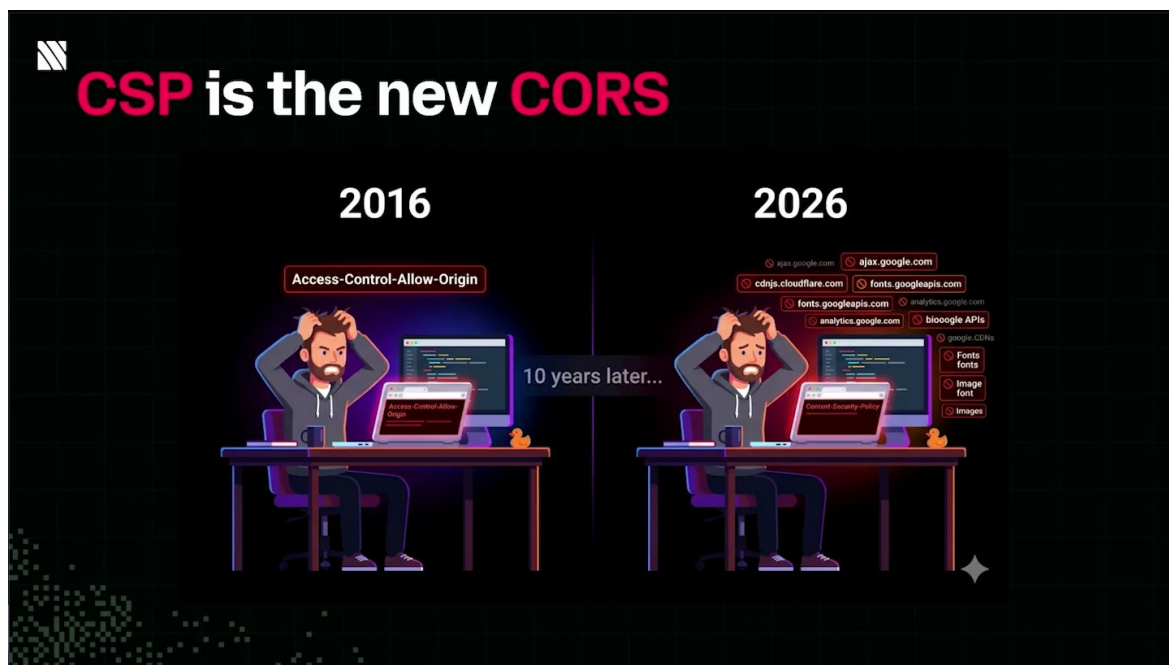


图 17: 讲者用“CSP is the new CORS”概括开发体验痛点：今天很多生产失败来自 CSP 域名声明遗漏。¹⁶

为什么类型安全也会影响 CSP 类型安全本身不能替代 CSP。它解决的是“工具结果和视图数据结构是否一致”的问题；CSP 解决的是“视图能访问哪些外部资源”的问题。二者互补：类型系统减少应用内部协议错误，CSP 限制应用外部资源边界，CSP Inspector 再把运行时访问记录和 metadata 声明做对账。

4.4 八球示例应用：从工具执行到视图渲染

讲者随后切到一个 Skybridge 新项目生成的示例代码库。这个示例应用是一个 magic eight-ball：用户提出问题，工具从 25 个预定义答案里选择一个回答。它有两个关键部分：一个 MCP tool 负责生成答案，一个 view 负责展示问题与答案。

¹⁶视频画面时间：00:14:49–00:15:24。

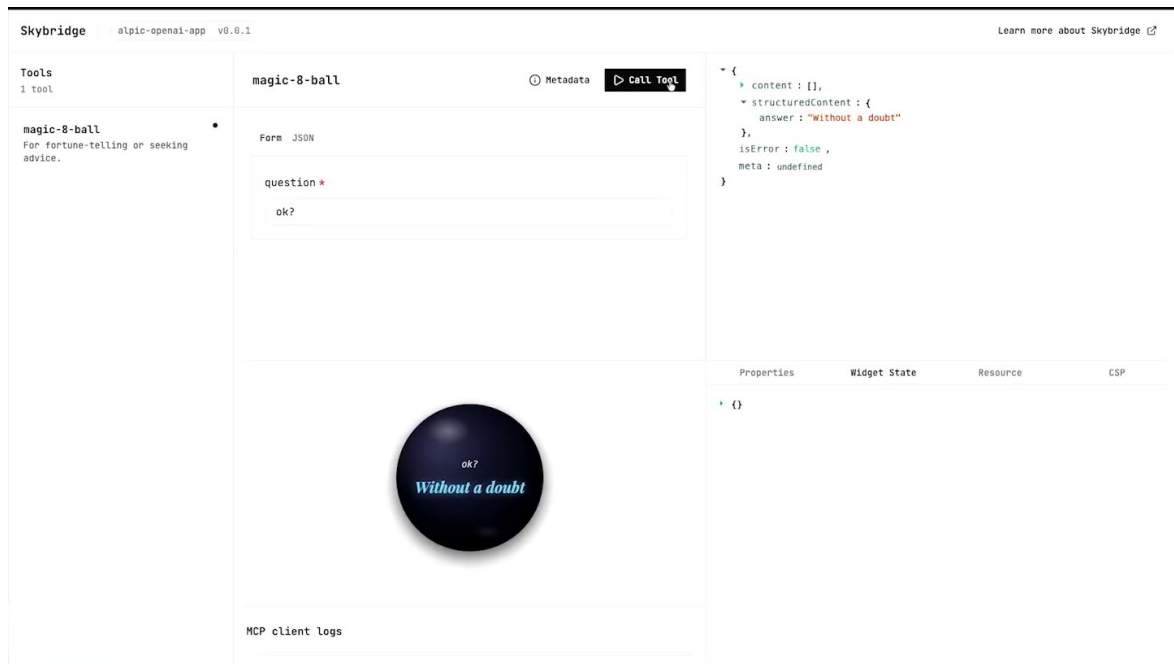


图 18: Skybridge Inspector 中的 eight-ball 示例：左侧是工具列表，中间是渲染出的 view，右侧显示 widget state 和工具结果。¹⁷

当 Skybridge server 启动后，浏览器里会出现一个开发工具页面。左侧列出应用暴露的工具；开发者可以直接执行其中一个工具，例如 magic eight-ball tool。若该工具关联了 view，开发工具就会把这个 view 渲染出来，方便开发者观察工具结果如何进入 UI，以及修改后是否能实时反映到页面上。

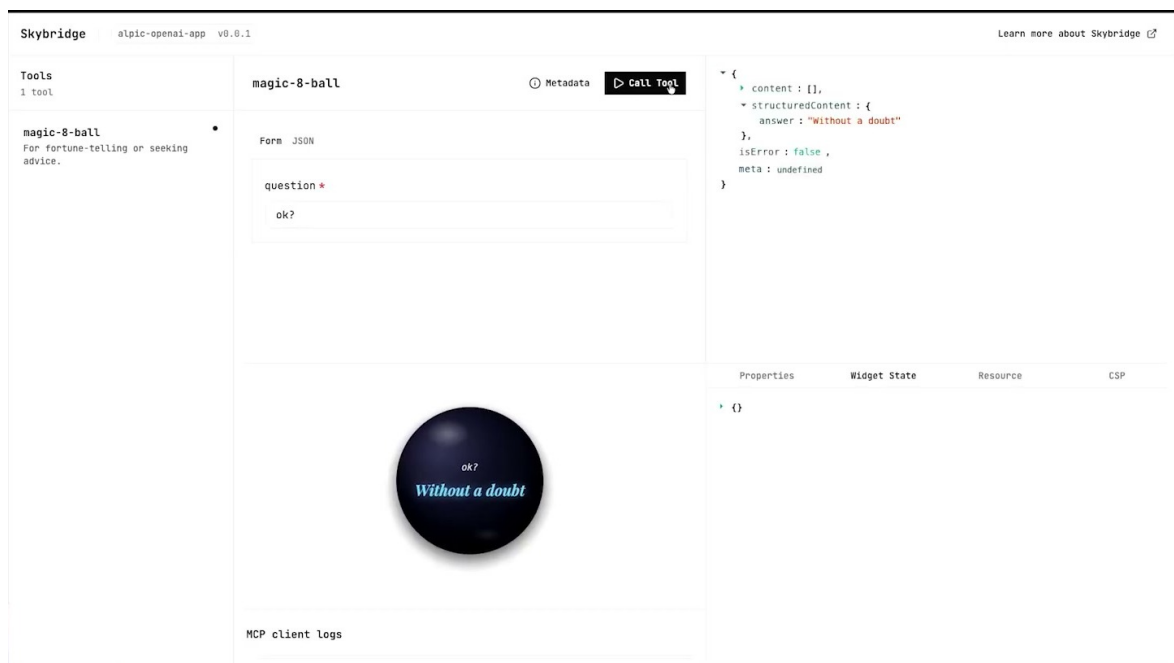


图 19: 本地 Inspector 把工具执行、工具结果和 view 渲染放在同一界面，开发者可以在提交到宿主前检查交互链路。¹⁸

¹⁷视频画面时间：00:17:41–00:18:04。

这个 demo 的意义在于把“宿主里才会发生的应用生命周期”提前搬到开发环境：工具被调用，结果生成，关联 view 被渲染，视图发起网络请求，CSP Inspector 记录实际访问域。开发者能在提交到 ChatGPT 之前看到更接近真实运行路径的反馈。

4.5 CSP Inspector demo：从缺失域名到绿色通过

CSP Inspector 的工作方式很朴素，也正因为朴素才有用：它读取开发者在 metadata 中声明的域名，同时观察 view 运行时实际访问的域名，然后比较两张清单。如果实际访问域名没有出现在 metadata 中，Inspector 就把它标成缺失项。

讲者的演示路径如下：

1. 初始状态下，Inspector 显示所有检查项为绿色，说明当前 view 的访问域都被 metadata 覆盖。
2. 开发者回到代码库，新增一次 API fetch，用来获取 IP location 信息。
3. 组件重新渲染后，Inspector 立刻记录到新增访问域，并把该域标记为 metadata 中缺失。
4. 开发者把这个缺失域名补进应用 metadata。
5. 刷新 Inspector 后，检查结果恢复为绿色。

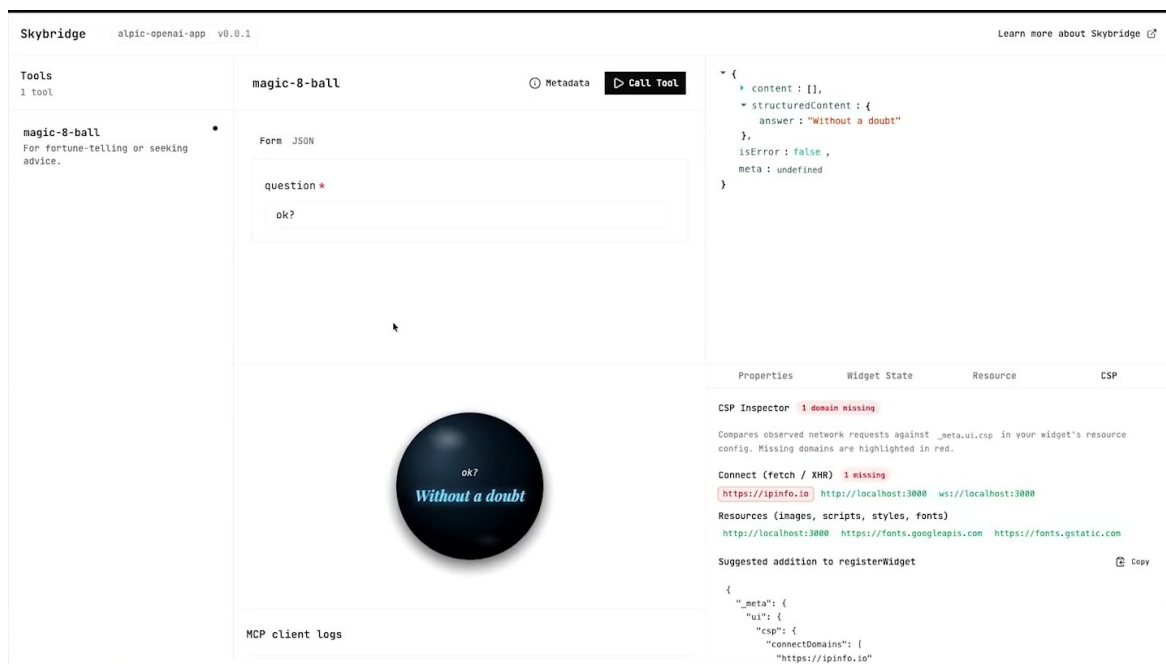


图 20: CSP Inspector 发现缺失域名：新增的 `https://ipinfo.io` 请求没有出现在 metadata 声明中，因此被标红。¹⁹

¹⁸视频画面时间：00:17:41–00:18:04。

¹⁹视频画面时间：00:18:30–00:18:52。

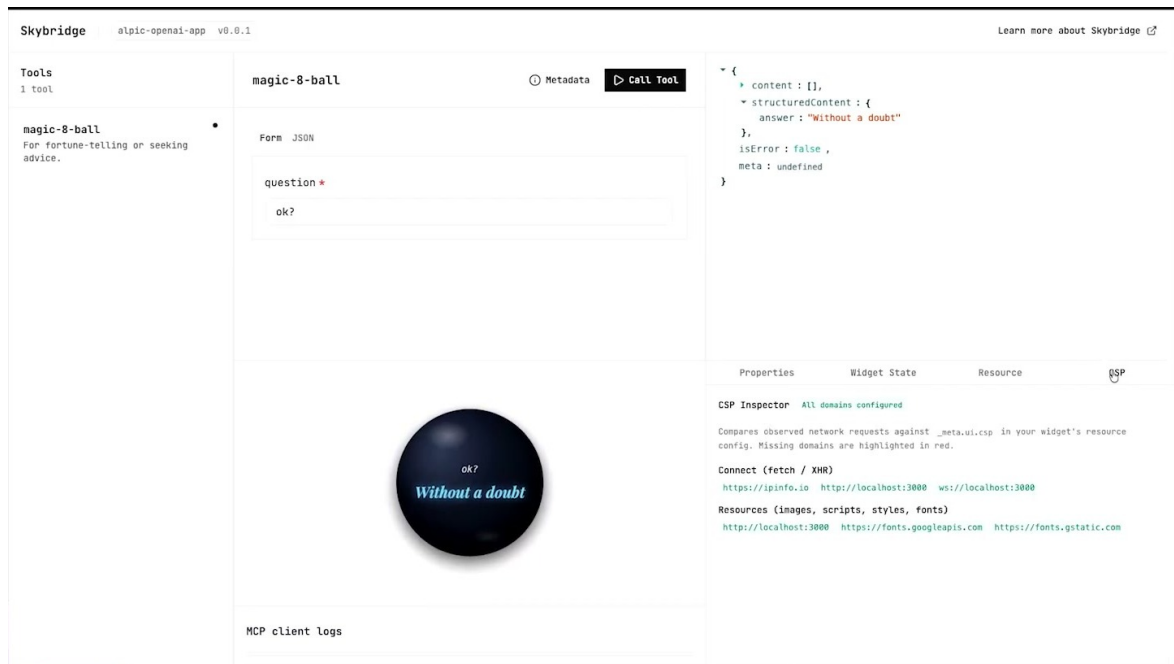


图 21: 补上缺失域名后, Inspector 重新显示绿色状态, 形成开发期的 detect-fix-verify 闭环。²⁰

这个流程指向非常具体的上线问题, 抽象安全概念只是背景。讲者明确说, 他们看到不少 Chat-GPT app store submission 因缺失 CSP 被拒, 也看到应用因为缺失 CSP domains 在生产环境失效。换句话说, CSP Inspector 的价值落在前置暴露故障: 它把“提交后才失败”的问题提前到开发期。

CSP Inspector 的核心闭环 CSP Inspector 把 CSP 调试变成对账流程: metadata 声明域名, view 实际访问域名, Inspector 比较二者并暴露缺口。这个闭环越早进入开发流程, 生产环境里因 connect-src、img-src、style-src 或 script-src 漏配导致的失败就越少。

4.6 本章小结

本章的重点是开发者侧的落地成本。双层 iframe、代理域、per-app subdomain 与 CSP meta injection 给宿主提供了安全边界; 开发者仍然需要把应用的外部依赖声明清楚。演讲者用 Skybridge 与 CSP Inspector 展示了一条更实用的路径: 用类型安全减少 MCP server 与 views 的数据错配, 用 host API polyfills 缓冲宿主差异, 用 Inspector 把 CSP 漏配提前暴露。对于构建者来说, 最现实的判断标准很简单: 应用在开发环境能跑, 只是第一步; 应用在带 CSP 的生产等价环境里能跑, 才接近可提交状态。

5 总结与延伸: 双层 iframe 的收益、成本与实践原则

讲者收尾时给了两类后续资源: 一类是 slides, 方便回看双层 iframe 与 CSP 的推导; 另一类是 Skybridge, 方便开发者直接试用工具链。抽奖和现场礼品属于活动信息, 对技术理解没有实质帮助, 整合时可以压缩成一句脚注或直接省略。

²⁰视频画面时间: 00:18:49-00:19:00。

5.1 产品原因：为什么对话产品要承载第三方 UI

整场演讲的起点是产品问题。MCP Apps 与 ChatGPT Apps 让第三方能力进入对话环境，用户可以在 chat 中发现应用、调用工具、看到交互式 view。纯文本结果无法覆盖所有产品体验：订票、选项比较、地图、表单、可视化结果、配置面板都需要 UI。

一旦宿主渲染第三方 UI，它就引入了三个资产保护问题：

- 宿主会话资产：cookies、localStorage、IndexedDB、用户登录状态；
- 宿主页面资产：DOM、对话内容、宿主脚本、宿主内部 API；
- 其他应用资产：另一个 app 的 origin-indexed storage 与运行状态。

产品想要开放生态，浏览器要求明确边界。双层 iframe 的价值就在这里：它让第三方 UI 能在对话里出现，同时把它从宿主 origin 中隔离出来。

5.2 浏览器机制：CSP、origin、sandbox 与双层 iframe

浏览器提供的基础材料并不神秘：iframe 创建嵌套 browsing context，origin 决定 DOM 与存储访问边界，sandbox 剥离或恢复能力，CSP 限制资源加载。麻烦来自这些机制的组合约束。

可以把前几章的机制压缩成下面的结构：

$$H \longrightarrow I_o \longrightarrow I_i, \quad O(I_i) = O_P, \quad O_P \neq O_H$$

其中：

- H ：宿主页面，例如 ChatGPT；
- I_o ：外层 iframe，来自代理受控域；
- I_i ：内层 iframe，承载应用 view；
- O_P ：代理受控 origin，通常可按 app 分配子域；
- O_H ：宿主 origin。

这个结构同时服务四件事：宿主只需要允许一个受控 `frame-src` 入口；第三方应用获得可用 origin 与运行环境；每个 app 通过子域隔离存储；应用内部 CSP 可以由 metadata 注入到内层上下文。

把双层 iframe 想成两道门 第一道门是宿主到代理域：它解决规模化嵌入问题，让宿主不必把所有第三方域名写进 `frame-src`。第二道门是代理域到应用 view：它解决应用代码运行问题，让 view 有可用 origin、可访问声明过的资源，并被应用级 CSP 约束。

5.3 安全边界：可加载、可存储、可通信都要受控

双层 iframe 的安全边界可以分成三层：

1. 资源边界：CSP 决定脚本、图片、样式、网络连接和 frame 来源。
2. 身份边界：origin 决定页面之间能否直接访问 DOM 和 origin-indexed storage。
3. 消息边界：postMessage 这类跨窗口通道决定 H 、 I_o 、 I_i 之间如何传递数据。

第三层经常被低估。浏览器的同源策略会挡住直接 DOM 访问，但消息通道是显式打开的门。只要存在 M 这样的 message channel，接收方就必须校验来源、消息类型、payload schema、一次性 token 或会话上下文。否则，攻击面会从 DOM 访问转移到 confused-deputy message handler。

postMessage 校验不能省，iframe 隔离只能限制直接访问，无法自动保证消息语义正确。每个 message handler 都应检查 `event.origin`、目标 frame 身份、消息类型、字段 schema 和授权上下文。把所有消息都当成可信输入，等于在隔离墙上开了无门禁通道。

5.4 运营规模：为什么代理域比无限域名清单更现实

如果每个应用都让宿主直接嵌入自己的外部域名，宿主的 `frame-src` 会变成无限增长的白名单。这个方案在少量内部应用里还勉强可控，进入 app store 模式后会迅速失控：新应用不断提交，旧应用变更域名，恶意应用可能尝试相似域名，审核系统需要持续追踪。

代理受控域把这个问题压缩成一个稳定入口。宿主允许 `https://*.openaiusercontent.com` 这类受控范围后，具体应用由代理层路由到 per-app subdomain。其工程收益是规模化：宿主 CSP 维持稳定，应用之间仍能通过子域隔离存储，审核和监控也有统一的基础设施入口。

这套设计也带来成本。代理层成为关键基础设施，路由、缓存、header 注入、CSP meta 生成、子域分配、日志与回滚都要可靠。代理 bug 可能把应用送到错误子域，或把 CSP 注入到错误上下文，后果会很严重。

5.5 开发者 workflow：从本地能跑到生产等价

对构建者来说，最实用的检查清单如下：

1. 列清 **origins**。明确宿主 origin O_H 、代理 origin O_P 、应用后端 origin、CDN origin、图片 origin、字体 origin、分析或错误上报 origin。
2. 写实 **metadata**。metadata 应覆盖 view 实际访问的域名，尤其是 `connect-src` 下的 API 域名。只写当前 demo 用到的域，后续新增依赖必须同步更新。
3. 收紧 **frame-src**。宿主层优先指向受控代理域；应用层需要 frame 时，再单独声明必要嵌入来源。
4. 分别检查资源指令。`connect-src` 管 API 与 WebSocket，`img-src` 管图片，`style-src` 管样式和字体关联路径，`script-src` 管脚本执行，`frame-src` 管嵌套 frame。

5. 确认 **sandbox flags**。每个 flag 都是在恢复能力；`allow-scripts`、`allow-same-origin`、`allow-forms`、`allow-popups` 应按真实需要启用。
6. 校验 **postMessage**。所有跨 frame 消息都要校验 `origin`、`source`、`type`、`schema` 和授权上下文。
7. 跑生产模式测试。关闭会掩盖 CSP 的 `developer mode`，使用接近真实宿主的 CSP、代理域、子域和 `sandbox` 组合进行验证。

可上线的最低条件 一个 MCP / ChatGPT app 的可上线状态可以概括为：

$$\text{ready} \Rightarrow \text{originSeparated} \wedge \text{cspCovered} \wedge \text{messagesValidated} \wedge \text{prodTested}$$

其中 `origin separation` 防止直接越界访问，`CSP coverage` 覆盖真实资源依赖，`message validation` 约束跨 frame 协议，`production-mode test` 验证开发环境没有掩盖关键失败。

5.6 开放风险：仍然容易踩坑的地方

双层 `iframe` 是强结构，仍然无法替开发者完成所有安全判断。几个风险值得单独记住。

- **过宽 CSP pattern**。`https://*.example.com` 这类模式很方便，也可能把测试域、用户内容域或可被接管的子域纳入信任范围。
- **消息验证松散**。只检查消息字段，忽略 `event.origin` 或 `event.source`，会让其他 frame 伪造合法操作。
- **代理路由 bug**。`per-app subdomain` 分配错误、缓存键错误、路径重写错误，都可能导致应用隔离失效或资源错配。
- **host-specific API 差异**。App SDK / App Extension 的共同规范之外，各宿主的 API、权限提示和生命周期事件可能不同。`polyfill` 能降低差异，无法消除差异。
- **dev/prod divergence**。`developer mode`、mock API、宽松本地域名和禁用 CSP 的调试环境会隐藏生产失败。

真正成熟的实践是把这些风险放进自动化流程：提交前执行 CSP Inspector，对所有 `declared domains` 与 `runtime domains` 做 diff；对 `message handler` 做 `schema` 测试；用生产等价 CSP 跑端到端测试；在审核前记录每个外部域名的用途和必要性。

5.7 本章小结

这场演讲的主线可以归结为一句工程判断：第三方 UI 要进入对话产品，宿主必须同时解决可用性、安全边界和规模化运营。单层 `iframe` 的直接方案会在 `nonce`、`same-origin`、`sandbox` 和 `frame-src` 上不断碰壁；双层 `iframe` 用代理域与内层 `srcdoc` 把问题拆开处理。代价同样明确：开发者必须维护准确 `metadata`，宿主必须维护可靠代理层，双方都要认真处理 CSP、`sandbox` 与 `postMessage`。Skybridge 和 CSP Inspector 的价值就在于把这些约束变成可见、可测、可修的开发流程，减少应用提交和生产环境中的低级失败。